EDA of Mobile Phones Dataset in Python using Jupyter Notebook

---

1. About the Author

Author Name: **Khawar Zaman**
Email: khawarzaman940@gmail.com

Github: https://github.com/dashboard\

LinkedIn: https://www.linkedin.com/feed/

2. What is and EDA?

`**Exploratory Data Analysis (EDA)**` is a fundamental process in data science that involves summarizing and visualizing data to uncover patterns,relationships,and key characteristics.The primary objective of EDA is to gain valuable insights,identify potential issues,and make informed decisions regarding the next steps in the data analysis workflow.EDA employs statistical and visualization techniques to analyze datasets,enabling the detection of missing values,outliers,trends,and distributions.By providing a comprehensive overview of the data,EDA ensures a deeper understanding before conducting more advanced statistical analyses or developing predictive models.This process is crucial for identifying anomalies,validating assumptions,and optimiz- ing data-driven decision-making in analytical projects.

2.1 What are the key steps in EDA?

EDA involves several key steps, including:

1. Understanding the Dataset

Before analyzing the data, it is essential to understand its structure, types of variables, and overall content.

A. Load the Data

- Read the dataset using Pandas (`pd.read_csv()` , `pd.read_excel()` , etc.).
- Connect to databases if data is stored in SQL or NoSQL formats.

B. Check Data Structure

- **Shape of Data**: Check the number of rows and columns using `.shape`.
- **Data Types**: Identify numerical, categorical, and datetime data using `.info()`.
- **Sample Records**: Use `.head()` and `.tail()` to view a snapshot of the dataset.

2. Handling Missing Values

Missing values can lead to incorrect conclusions, so handling them appropriately is crucial.

A. Detect Missing Values

- Use `.isnull().sum()` to check for missing values.
- Visualize missing data using **heatmaps** (Seaborn's `sns.heatmap`).

B. Handle Missing Data

- **Remove Missing Values**: Drop rows or columns with too many missing values.
- **Imputation**: Replace missing values using:
  - **Mean/Median/Mode** for numerical data.
  - **Most frequent category** for categorical data.
  - **Interpolation** for time-series data.

3. Handling Outliers

Outliers can distort the data distribution and affect model performance.

A. Detect Outliers

- **Box Plots**: Identify extreme values.
- **Z-Score**: Detect values that are far from the mean (`scipy.stats.zscore`).
- **Interquartile Range (IQR)**: Remove data points beyond Q1 - 1.5*IQR or Q3 + 1.5*IQR.

B. Handle Outliers

- **Remove outliers** if they are due to data entry errors.
- **Transform data** (log transformation, winsorization) if outliers are genuine but affect analysis.

4. Univariate Analysis (Analyzing Single Variables)

Univariate analysis helps understand the distribution of individual variables.

A. Numerical Variables

- **Histogram**: Shows distribution of values.
- **Box Plot**: Identifies outliers.
- **KDE Plot**: Displays the probability density function.

B. Categorical Variables

- **Bar Chart**: Displays frequency distribution.
- **Pie Chart**: Shows proportions (use with caution, as bar charts are often more effective).

5. Bivariate and Multivariate Analysis

This step examines relationships between variables.

A. Bivariate Analysis (Two Variables)

- **Numerical vs. Numerical**
  - **Scatter Plot**: Identifies correlation trends.
  - **Correlation Coefficient (Pearson, Spearman, Kendall)**: Measures relationship strength.
- **Numerical vs. Categorical**
  - **Box Plot**: Compares distributions across categories.
  - **Violin Plot**: Displays density and spread of values.
- **Categorical vs. Categorical**
  - **Crosstab & Stacked Bar Charts**: Compare frequencies.
  - **Chi-Square Test**: Determines statistical dependence.

B. Multivariate Analysis (More Than Two Variables)

- **Pair Plot (Seaborn `sns.pairplot`)**: Displays pairwise relationships.
- **Heatmap (`sns.heatmap`)**: Shows correlation among multiple variables.

6. Feature Engineering & Transformation

Feature engineering helps create more meaningful features for modeling.

A. Encoding Categorical Data

- **One-Hot Encoding**: Converts categorical values into binary columns (`pd.get_dummies()`).
- **Label Encoding**: Assigns numerical labels to categories (`LabelEncoder` in `sklearn`).

B. Scaling & Normalization

- **Standardization**: Transforms data to have mean = 0 and standard deviation = 1.
- **Min-Max Scaling**: Scales values between 0 and 1 (`MinMaxScaler` in `sklearn`).

C. Feature Extraction & Creation

- **Date Features**: Extract day, month, year from timestamps.
- **Polynomial Features**: Create new features by combining existing ones.

7. Dimensionality Reduction (Optional, for Large Datasets)

If a dataset has many variables, reducing dimensionality helps simplify analysis.

- **Principal Component Analysis (PCA)**: Reduces high-dimensional data while retaining variance.
- **t-SNE & UMAP**: Used for visualization of high-dimensional data.

8. Data Visualization

Visualization helps communicate insights effectively.

- **Matplotlib & Seaborn** for static plots.
- **Plotly & Dash** for interactive visualizations.

9. Summary and Insights

Finally, summarize key findings from EDA:

- **Data quality issues (missing values, outliers, skewness).**
- **Important relationships and trends.**
- **Potential feature transformations needed before modeling.**

---

3. About the Data

3.1 Dataset

[source] kaagle: https://www.kaggle.com/datasets/arifmia/mobile-phones-dataset-specs-prices-and-trends\ [Author] Profile link : https://www.kaggle.com/arifmia

3.2 📌 Overview

This dataset provides detailed specifications and official launch prices of various smartphone models from different manufacturers. It offers valuable insights into smartphone hardware, pricing trends, and brand competitiveness across multiple countries. One of the key aspects of this dataset is the pricing information, which represents the official launch prices of mobile phones at the time of their release. Since prices vary based on region and launch year, this dataset can be useful for analyzing price trends over time and comparing smartphone affordability across different markets.

3.3 📊 Features Included

- **Company Name** – The brand or manufacturer of the mobile phone.
- **Model Name** – The specific model of the smartphone.
- **Mobile Weight** – The weight of the smartphone (in grams).
- **RAM** – The amount of Random Access Memory (RAM) in GB.
- **Front Camera** – The resolution of the front (selfie) camera (in MP).
- **Back Camera** – The resolution of the primary rear camera (in MP).
- **Processor** – The chipset or processor used in the device.
- **Battery Capacity** – The battery size of the smartphone (in mAh).
- **Screen Size** – The display size of the smartphone (in inches).
- **Launched Price (Pakistan, India, China, USA, Dubai)** – The official launch price of the mobile phone in various countries at the time of release.

3.4 🔍 Potential Use Cases

- **Price Prediction** – Build a machine learning model to predict smartphone prices based on specifications.
- **Trend Analysis** – Study how smartphone prices and specifications have evolved over time.
- **Brand Competitiveness** – Compare different brands based on price, specifications, and market strategy.
- **Regional Price Comparison** – Analyze price variations across different countries.

3.5 📌 Why This Dataset?

- Includes launch prices from multiple countries, providing a global perspective.
- Covers various smartphone features, making it suitable for both price analysis and feature comparison.
- Useful for both data science projects and market research in the mobile industry.

---

4. Tasks of doing EDA

The purpose of doing the eda is to explore the data, understand the distribution of the data, and identify the relationship between the variables. Finding the problem is the data and resolving all the probelm with like missing values and outliers in the data,and make the inform features engineering or model choices.The goal is to transfrom the data in a way that is suitable for furthur analysis and get the valuable insights form the data.

---

5. Objectives of doing EDA

The objectives of doing the EDA is to explore the data in a way to get the information regarding the colomns or variables in data, understand the distribution of data in the colomns, finding the realtionsships between the colomns that how the colomns are realated to each other,repalcing the missing values and remove the outliers form the data. In the last we are answer the question realted to the data and apply any type of teachnique to the data if it is given in the tast.

Now before going towords the data, first we have to install the necessary libraries, and than we will call the dataset.

6. Import the Necessary Libraries

In [347…

```
# import the necessary library
import pandas as pd                 # for data manipulation
import numpy as np                  # for numerical computation
import matplotlib.pyplot as plt     # for data visualization
import seaborn as sns               # for data visualization
```

---

7. Load the Mobile Phone Dataset

In [348…

```
# Load the Mobile Phone dataset
df=pd.read_excel("Mobile_Dataset.xlsx")
df.head()
```

Out[348...

	Company Name	Model Name	Mobile Weight	RAM	Front Camera	Back Camera	Processor	Battery Capacity	Screen Size	Laun (Paki
0	Apple	iPhone 16 128GB	174g	6GB	12MP	48MP	A17 Bionic	3,600mAh	6.1 inches	224
1	Apple	iPhone 16 256GB	174g	6GB	12MP	48MP	A17 Bionic	3,600mAh	6.1 inches	234
2	Apple	iPhone 16 512GB	174g	6GB	12MP	48MP	A17 Bionic	3,600mAh	6.1 inches	244
3	Apple	iPhone 16 Plus 128GB	203g	6GB	12MP	48MP	A17 Bionic	4,200mAh	6.7 inches	249
4	Apple	iPhone 16 Plus 256GB	203g	6GB	12MP	48MP	A17 Bionic	4,200mAh	6.7 inches	259

---

8. Seeing the types of varaibles or variable info , name of variables , no. of rows and colomsn dataset

In [349...

```
# Check the types of variables in the df
df.dtypes
```

Out[349...

```
Company Name                 object
Model Name                   object
Mobile Weight                object
RAM                          object
Front Camera                 object
Back Camera                  object
Processor                    object
Battery Capacity             object
Screen Size                  object
Launched Price (Pakistan)    object
Launched Price (India)       object
Launched Price (China)       object
Launched Price (USA)         object
Launched Price (Dubai)       object
Launched Year                 int64
dtype: object
```

In [350…
```
# Check the information of the df dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 930 entries, 0 to 929
Data columns (total 15 columns):
 #   Column                      Non-Null Count  Dtype 
---  ------                      --------------  ----- 
 0   Company Name                930 non-null    object
 1   Model Name                  930 non-null    object
 2   Mobile Weight               930 non-null    object
 3   RAM                         930 non-null    object
 4   Front Camera                930 non-null    object
 5   Back Camera                 930 non-null    object
 6   Processor                   930 non-null    object
 7   Battery Capacity            930 non-null    object
 8   Screen Size                 930 non-null    object
 9   Launched Price (Pakistan)   930 non-null    object
 10  Launched Price (India)      930 non-null    object
 11  Launched Price (China)      930 non-null    object
 12  Launched Price (USA)        930 non-null    object
 13  Launched Price (Dubai)      930 non-null    object
 14  Launched Year               930 non-null    int64 
dtypes: int64(1), object(14)
memory usage: 109.1+ KB
```

In [351…
```
# Check the no. of rows and colomns in the df
df.shape
```

Out[351…
```
(930, 15)
```

In [352…
```
# Check the colomns names in df
df.columns
```

Out[352…
```
Index(['Company Name', 'Model Name', 'Mobile Weight', 'RAM', 'Front Camera',
       'Back Camera', 'Processor', 'Battery Capacity', 'Screen Size',
       'Launched Price (Pakistan)', 'Launched Price (India)',
       'Launched Price (China)', 'Launched Price (USA)',
       'Launched Price (Dubai)', 'Launched Year'],
      dtype='object')
```

Note That

Now from the above, we have seen the following:

1. All the colomns in the data have a type of object ,but we see that we have colomns with the numeric entries along with the string in it, which is not good. We need to convert the numeric entries to numeric type. We can do this later in this file.
2. Secondly, we have total of 930 entries in each colomn and no. of colomns are 15 in number.
3. In the last we see the names of colomns or variables in df dataset.

9. Now check the summary of df dataset, missing values in the df dataset and duplicated entries in df dataset.

In [353…
```
# Missing valus in df
df.isnull().sum()
```

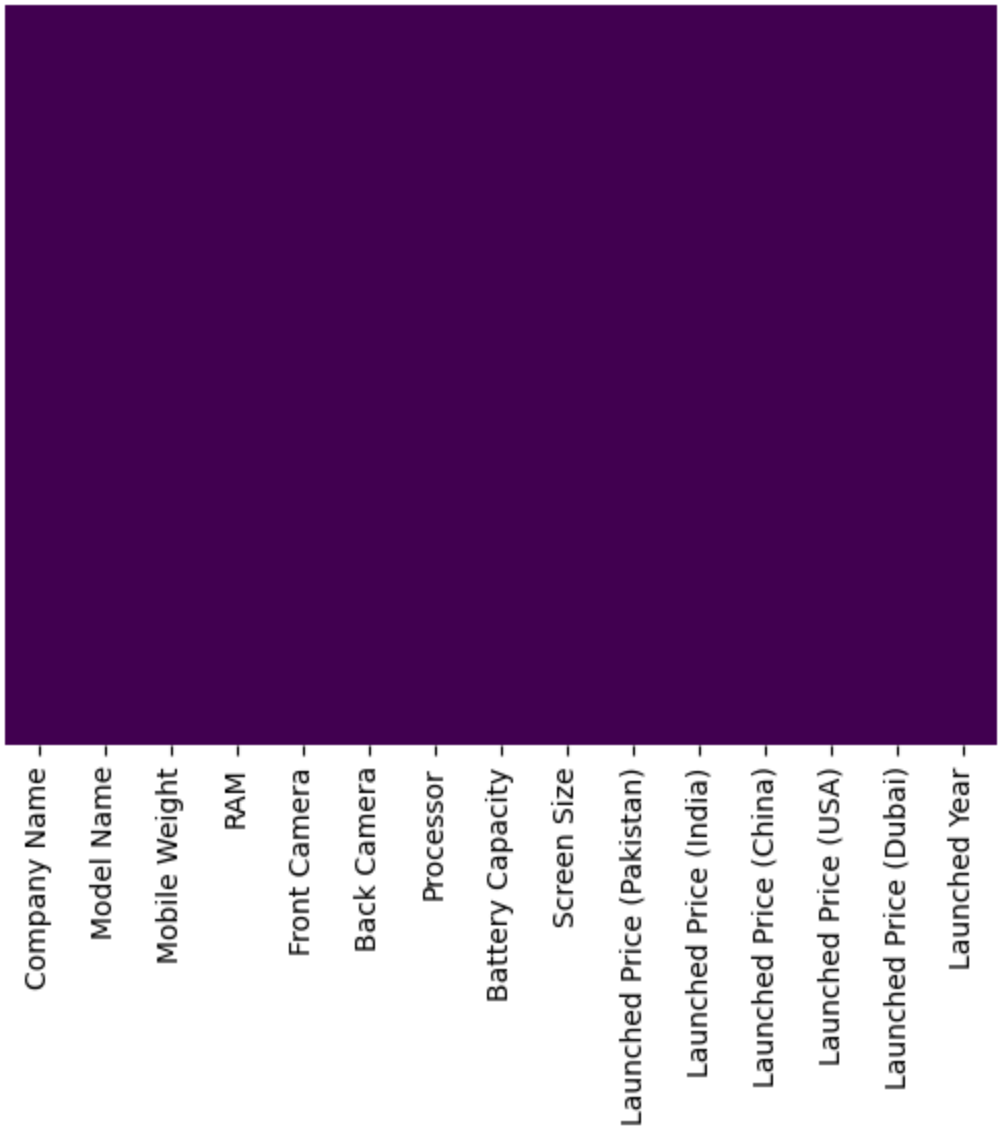
Out[353…
```
Company Name                0
Model Name                  0
Mobile Weight               0
RAM                         0
Front Camera                0
Back Camera                 0
Processor                   0
Battery Capacity            0
Screen Size                 0
Launched Price (Pakistan)   0
Launched Price (India)      0
Launched Price (China)      0
Launched Price (USA)        0
Launched Price (Dubai)      0
Launched Year               0
dtype: int64
```

In [354…
```
# See the missing values in terms of percentages
df.isnull().sum()/len(df)*100
```

Out[354…
```
Company Name                0.0
Model Name                  0.0
Mobile Weight               0.0
RAM                         0.0
Front Camera                0.0
Back Camera                 0.0
Processor                   0.0
Battery Capacity            0.0
Screen Size                 0.0
Launched Price (Pakistan)   0.0
Launched Price (India)      0.0
Launched Price (China)      0.0
Launched Price (USA)        0.0
Launched Price (Dubai)      0.0
Launched Year               0.0
dtype: float64
```

In [355…
```
# Check the missing values in df using heatmap
sns.heatmap(df.isnull(),yticklabels=False,cbar=False,cmap='viridis')
```

Out[355…
```
<Axes: >
```



In [356…
```
# Duplicates values in df
df.duplicated().sum()
```

Out[356…
```
np.int64(15)
```

In [357…
```
# The summary of the df
df.describe()
```

Out[357…

	Launched Year
count	930.000000
mean	2023.161290
std	29.629971
min	2014.000000
25%	2021.000000
50%	2023.000000
75%	2024.000000
max	2924.000000

Note That

1. In the df dataset there is no missing values in any colomn.
2. There are 15 duplicate entries in the df dataset.
3. We see the summary of the df dataset, that most all the colomns are of type object, and one colomn model year is of `type int64`. We will resolve this problem later in this file.

- **As we see that there is no missing value in the dataset, but if there is than first we see the colomn data type(the colomn is numeric replace it with mean and median, if the colomn is categorical variable then replace it with mode) and then replace it inplace of missing value.**

---

10. Feature Engineerng or Variable Transformation of variable or colomns in df dataset

We see in the data that most of the colomns are numeric and some are categorical.We need to transform categorical variables into numeric variables so that we can use them for further analysis.The colomns `Mobile Weight, RAM, Front Camera, Back Camera, Battery Capacity, Screen Size, Launched Price(Pakistan),Launched Price(India),Launched Price(China),Launched Price(USA),Launched Price(Dubai)`

I. First we check the issue with the `Mobile weight` colomn in df dataset, and than make the colomn numeric.

In [358…

```
df['Mobile Weight'].unique()
```

Out[358…

```
array(['174g', '203g', '206g', '221g', '171g', '172g', '140g', '204g',
       '238g', '135g', '164g', '189g', '228g', '194g', '188g', '226g',
       '177g', '208g', '458g', '490g', '300.5g', '468g', '682g', '708g',
       '234g', '196g', '168g', '195g', '167g', '254g', '187g', '263g',
       '199g', '190g', '202g', '198g', '192g', '235g', '191g', '181g',
       '178g', '175g', '165g', '143g', '229g', '732g', '586g', '498g',
       '523g', '726g', '567g', '503g', '480g', '366g', '508g', '433g',
       '674g', '426g', '360g', '205g', '179g', '200g', '183g', '185g',
       '180g', '173g', '184g', '170g', '162g', '210g', '155g', '215g',
       '550g', '610g', '223g', '150g', '146g', '158g', '163g', '153g',
       '176g', '201g', '193g', '159g', '156g', '182g', '535g', '510g',
       '500g', '560g', '520g', '540g', '555g', '239g', '186g', '533g',
       '218g', '207g', '233g', '222.8g', '440g', '482g', '466g', '499g',
       '372g', '465g', '209g', '166g', '169g', '295g', '255g', '197g',
       '225g', '220g', '241g', '245g', '161g', '580g', '147g', '151g',
       '212g', '213g', '216g', '222g', '250g', '450g', '420g', '280g',
       '230g', '240g', '470g', '211g', '227g', '236g', '242g', '288g',
       '219g', '267g', '231g', '460g', '485g', '530g', '495g', '590g',
       '505g', '475g', '178.8g', '571g'], dtype=object)
```

In [359…

```
df['Mobile Weight'].value_counts()
```

Out[359…

Mobile	Weight
190g	68
195g	64
185g	29
192g	26
180g	25
186g	22
205g	22
198g	22
206g	21
188g	21
189g	20
191g	20
210g	18
199g	18
203g	17
196g	16
184g	16
194g	15
168g	15
172g	15
202g	15
179g	14
173g	13
221g	13
174g	12
175g	12
182g	12
209g	12
187g	11
200g	11
193g	11
181g	10
228g	9
183g	8
177g	8
204g	8
170g	7
171g	6
220g	6
460g	6
164g	6
234g	5
208g	5
207g	5
239g	5
708g	5
218g	4
223g	4
226g	4
238g	4
219g	4
450g	4
201g	4
550g	4
215g	4

155g	4
520g	4
213g	4
197g	4
440g	4
167g	4
178g	4
140g	3
135g	3
230g	3
490g	3
480g	3
162g	3
682g	3
466g	3
586g	3
468g	3
530g	3
458g	2
159g	2
508g	2
235g	2
143g	2
263g	2
254g	2
300.5g	2
366g	2
146g	2
610g	2
231g	2
485g	2
590g	2
571g	2
211g	2
372g	2
499g	2
482g	2
580g	2
420g	2
225g	2
560g	2
222.8g	2
533g	2
510g	2
176g	2
498g	2
465g	2
166g	2
169g	2
245g	2
156g	2
674g	1
426g	1
523g	1
165g	1
732g	1

```
229g       1
567g       1
726g       1
503g       1
433g       1
163g       1
158g       1
153g       1
150g       1
360g       1
500g       1
555g       1
233g       1
241g       1
255g       1
295g       1
535g       1
540g       1
222g       1
147g       1
161g       1
470g       1
240g       1
250g       1
280g       1
151g       1
212g       1
216g       1
227g       1
267g       1
236g       1
242g       1
288g       1
495g       1
505g       1
475g       1
178.8g     1
Name: count, dtype: int64
```

In [360…

```
# now i have seen that the entries in the Mobile Weighs colomn has g in it so we ne
df['Mobile Weight']=df['Mobile Weight'].str.replace('g','').astype(float)
```

In [361…

```
# Rename the Mobile Weight colomn to Mobile Weight(g)
df.rename(columns={'Mobile Weight':'Mobile Weight(g)'},inplace=True)
df.head()
```

Out[361...

	Company Name	Model Name	Mobile Weight(g)	RAM	Front Camera	Back Camera	Processor	Battery Capacity	Screen Size	La (Pa
0	Apple	iPhone 16 128GB	174.0	6GB	12MP	48MP	A17 Bionic	3,600mAh	6.1 inches	
1	Apple	iPhone 16 256GB	174.0	6GB	12MP	48MP	A17 Bionic	3,600mAh	6.1 inches	
2	Apple	iPhone 16 512GB	174.0	6GB	12MP	48MP	A17 Bionic	3,600mAh	6.1 inches	
3	Apple	iPhone 16 Plus 128GB	203.0	6GB	12MP	48MP	A17 Bionic	4,200mAh	6.7 inches	
4	Apple	iPhone 16 Plus 256GB	203.0	6GB	12MP	48MP	A17 Bionic	4,200mAh	6.7 inches	

II. First we check the issue with the `RAM` colomn in df dataset, and than make the colomn numeric.

In [362...
```
df['RAM'].unique()
```

Out[362...
```
array(['6GB', '8GB', '4GB', '3GB', '12GB', '2GB', '1.5GB', '16GB', '10GB',
       '1GB', '8GB / 12GB'], dtype=object)
```

In [363...
```
df['RAM'].value_counts()
```

Out[363...
```
RAM
8GB           308
6GB           206
12GB          193
4GB           146
3GB            34
16GB           31
2GB             6
1.5GB           2
8GB / 12GB      2
10GB            1
1GB             1
Name: count, dtype: int64
```

Now i have seen that the entries in the RAM colomn has GB in it, also two to the entries with the two entries like this 8GB / 12GB we need to remove it to make it numeric.

In [364…
```python
# Function to clean RAM values
def clean_ram(value):
    if isinstance(value, str):
        value = value.replace('GB', '').strip()  # Remove 'GB' and trim spaces
        if '/' in value:  # Handle cases like '8 / 12'
            numbers = [int(x) for x in value.split('/') if x.strip().isdigit()]
            return sum(numbers) / len(numbers)  # Taking the average
        return float(value)  # Convert to float
    return value  # Return the value as is if it's not a string

# Apply cleaning function
df['RAM'] = df['RAM'].apply(clean_ram)
```

In [365…
```python
# now replace the name RAM with RAM(GB)
df.rename(columns={'RAM':'RAM(GB)'},inplace=True)
```

In [366…
```python
df['RAM(GB)'].value_counts()
```

Out[366…
```
RAM(GB)
8.0     308
6.0     206
12.0    193
4.0     146
3.0      34
16.0     31
2.0       6
10.0      3
1.5       2
1.0       1
Name: count, dtype: int64
```

III. First we check the issue with the `Battery Capacity` colomn in df dataset, and than make the colomn numeric.

In [367…
```python
df['Battery Capacity'].unique()
```

Out[367…

```
array(['3,600mAh', '4,200mAh', '4,400mAh', '4,500mAh', '3,200mAh',
       '4,300mAh', '4,325mAh', '2,438mAh', '3,240mAh', '3,095mAh',
       '4,352mAh', '2,227mAh', '2,815mAh', '3,687mAh', '3,110mAh',
       '3,046mAh', '3,969mAh', '2,716mAh', '2,658mAh', '3,174mAh',
       '2,942mAh', '7,608mAh', '8,612mAh', '5,124mAh', '7,812mAh',
       '9,720mAh', '10,307mAh', '5000mAh', '4800mAh', '4000mAh',
       '4700mAh', '3900mAh', '4500mAh', '3800mAh', '4400mAh', '3700mAh',
       '6000mAh', '4300mAh', '3500mAh', '4050mAh', '3000mAh', '3600mAh',
       '3300mAh', '2600mAh', '11200mAh', '10090mAh', '8400mAh', '8000mAh',
       '7040mAh', '5100mAh', '5050mAh', '7600mAh', '4510mAh', '4115mAh',
       '4085mAh', '9510mAh', '11000mAh', '5700mAh', '4100mAh', '3315mAh',
       '3260mAh', '2300mAh', '3055mAh', '4030mAh', '2000mAh', '3200mAh',
       '4200mAh', '4450mAh', '4600mAh', '4830mAh', '4870mAh', '8040mAh',
       '4805mAh', '5800mAh', '5600mAh', '6400mAh', '8360mAh', '4520mAh',
       '5,000mAh', '4,600mAh', '4,700mAh', '4,350mAh', '4,000mAh',
       '4,025mAh', '4,040mAh', '5,100mAh', '6,500mAh', '5,500mAh',
       '6,000mAh', '5,800mAh', '5,200mAh', '7,100mAh', '8,360mAh',
       '8,340mAh', '6,400mAh', '7,200mAh', '6,100mAh', '5,400mAh',
       '4,610mAh', '5,110mAh', '4,050mAh', '3,350mAh', '3,500mAh',
       '4,310mAh', '3,800mAh', '4,020mAh', '4,100mAh', '4,360mAh',
       '4,460mAh', '4,815mAh', '4,800mAh', '4,750mAh', '4,900mAh',
       '5,050mAh', '8200mAh', '10,100mAh', '3,000mAh', '3,700mAh',
       '2,800mAh', '3,140mAh', '3,885mAh', '4,080mAh', '4,680mAh',
       '4,614mAh', '5,003mAh', '4,410mAh', '4,355mAh', '4,385mAh',
       '4,575mAh', '5,250mAh', '8,000mAh', '7,500mAh', '7,000mAh',
       '9,000mAh', '5,300mAh', '3,750mAh', '5,450mAh', '5,550mAh',
       '7,250mAh', '8,500mAh', '8,300mAh', '10,000mAh', '10,500mAh',
       '8,850mAh', '5160mAh', '5065mAh', '5110mAh'], dtype=object)
```

In [368…

```
df['Battery Capacity'].value_counts()
```

Out[368…

Battery Capacity	
5,000mAh	197
5000mAh	96
4,500mAh	46
4500mAh	38
5,200mAh	35
4,000mAh	24
6000mAh	20
4,300mAh	20
5,100mAh	19
5,500mAh	19
6,000mAh	15
4,600mAh	14
4000mAh	12
4300mAh	10
8040mAh	9
4,800mAh	9
4800mAh	8
4,400mAh	8
4,200mAh	8
5,050mAh	8
3700mAh	8
5100mAh	7
4700mAh	7
4400mAh	7
4,700mAh	7
4200mAh	6
4600mAh	6
7,100mAh	6
4,025mAh	6
3,200mAh	6
9510mAh	6
2,815mAh	6
7,250mAh	5
10,307mAh	5
3300mAh	5
3,500mAh	4
5160mAh	4
4,020mAh	4
5,800mAh	4
3000mAh	4
3,110mAh	3
3,969mAh	3
3,046mAh	3
4,325mAh	3
3,095mAh	3
3,240mAh	3
2,438mAh	3
3,600mAh	3
3,687mAh	3
2,227mAh	3
4,352mAh	3
2,942mAh	3
2,658mAh	3
6,100mAh	3
5,110mAh	3

4,050mAh	3
7,812mAh	3
5800mAh	3
4100mAh	3
4050mAh	3
9,720mAh	3
3500mAh	3
8,000mAh	3
9,000mAh	3
10,000mAh	3
10,500mAh	3
8,500mAh	3
3,174mAh	3
2,716mAh	2
4830mAh	2
4,350mAh	2
6400mAh	2
8000mAh	2
11200mAh	2
7040mAh	2
5,124mAh	2
8,612mAh	2
4450mAh	2
5050mAh	2
10090mAh	2
3900mAh	2
7,608mAh	2
3800mAh	2
5,300mAh	2
5,550mAh	2
4,385mAh	2
3,700mAh	2
4,100mAh	2
4,815mAh	2
10,100mAh	2
4,310mAh	2
4,750mAh	2
4,610mAh	2
7,200mAh	2
5,400mAh	2
8,360mAh	2
8,340mAh	2
6,400mAh	2
4870mAh	2
4520mAh	2
8360mAh	2
3200mAh	2
3260mAh	2
4510mAh	2
4,900mAh	2
3,800mAh	2
6,500mAh	2
5700mAh	1
11000mAh	1
2000mAh	1
4030mAh	1

```
8400mAh        1
3600mAh        1
2600mAh        1
7600mAh        1
4805mAh        1
5600mAh        1
4115mAh        1
3315mAh        1
3055mAh        1
2300mAh        1
4085mAh        1
4,040mAh       1
8200mAh        1
4,360mAh       1
4,460mAh       1
3,350mAh       1
3,140mAh       1
3,885mAh       1
3,000mAh       1
2,800mAh       1
5,250mAh       1
4,575mAh       1
4,410mAh       1
4,355mAh       1
5,003mAh       1
4,614mAh       1
4,080mAh       1
4,680mAh       1
3,750mAh       1
5,450mAh       1
7,000mAh       1
7,500mAh       1
8,300mAh       1
8,850mAh       1
5065mAh        1
5110mAh        1
Name: count, dtype: int64
```

In [369…

```
# Now we have to remove the mAh from the Battery Capacity colomn along with removin
df['Battery Capacity']=df['Battery Capacity'].str.replace('mAh','').str.replace(','
```

In [370…

```
df['Battery Capacity'].value_counts()
```

Out[370…

Battery	Capacity
5000.0	293
4500.0	84
4000.0	36
6000.0	35
5200.0	35
4300.0	30
5100.0	26
4600.0	20
5500.0	19
4800.0	17
4400.0	15
4700.0	14
4200.0	14
3700.0	10
5050.0	10
8040.0	9
3200.0	8
3500.0	7
5800.0	7
9510.0	6
7100.0	6
4025.0	6
2815.0	6
4050.0	6
4100.0	5
3300.0	5
7250.0	5
10307.0	5
3000.0	5
8000.0	5
6400.0	4
4020.0	4
8360.0	4
5110.0	4
3600.0	4
3800.0	4
5160.0	4
3046.0	3
2438.0	3
3969.0	3
10500.0	3
8500.0	3
6100.0	3
9000.0	3
10000.0	3
3110.0	3
9720.0	3
2942.0	3
3174.0	3
7812.0	3
2658.0	3
2227.0	3
4352.0	3
3095.0	3
3687.0	3

4325.0	3
3240.0	3
5124.0	2
4385.0	2
5550.0	2
4900.0	2
10100.0	2
8340.0	2
4870.0	2
6500.0	2
5300.0	2
4750.0	2
7040.0	2
3260.0	2
4510.0	2
3900.0	2
11200.0	2
10090.0	2
8612.0	2
2716.0	2
7608.0	2
4310.0	2
7200.0	2
4520.0	2
4350.0	2
4450.0	2
4830.0	2
4610.0	2
4815.0	2
5400.0	2
2600.0	1
4085.0	1
8400.0	1
7600.0	1
4115.0	1
2300.0	1
3315.0	1
11000.0	1
5700.0	1
4460.0	1
3055.0	1
5600.0	1
4805.0	1
2000.0	1
4360.0	1
3350.0	1
4040.0	1
4030.0	1
8200.0	1
3140.0	1
2800.0	1
4575.0	1
4355.0	1
4410.0	1
5003.0	1
4680.0	1

```
4614.0       1
4080.0       1
3885.0       1
5450.0       1
3750.0       1
7500.0       1
7000.0       1
5250.0       1
8300.0       1
8850.0       1
5065.0       1
Name: count, dtype: int64
```

In [371…
```
# Rename the Battery Capacity colomn to Battery Capacity(mAh)
df.rename(columns={'Battery Capacity':'Battery Capacity(mAh)'},inplace=True)
```

In [372…
```
df['Battery Capacity(mAh)'].value_counts()
```

Out[372…

Battery Capacity(mAh)	
5000.0	293
4500.0	84
4000.0	36
6000.0	35
5200.0	35
4300.0	30
5100.0	26
4600.0	20
5500.0	19
4800.0	17
4400.0	15
4700.0	14
4200.0	14
3700.0	10
5050.0	10
8040.0	9
3200.0	8
3500.0	7
5800.0	7
9510.0	6
7100.0	6
4025.0	6
2815.0	6
4050.0	6
4100.0	5
3300.0	5
7250.0	5
10307.0	5
3000.0	5
8000.0	5
6400.0	4
4020.0	4
8360.0	4
5110.0	4
3600.0	4
3800.0	4
5160.0	4
3046.0	3
2438.0	3
3969.0	3
10500.0	3
8500.0	3
6100.0	3
9000.0	3
10000.0	3
3110.0	3
9720.0	3
2942.0	3
3174.0	3
7812.0	3
2658.0	3
2227.0	3
4352.0	3
3095.0	3
3687.0	3

4325.0	3
3240.0	3
5124.0	2
4385.0	2
5550.0	2
4900.0	2
10100.0	2
8340.0	2
4870.0	2
6500.0	2
5300.0	2
4750.0	2
7040.0	2
3260.0	2
4510.0	2
3900.0	2
11200.0	2
10090.0	2
8612.0	2
2716.0	2
7608.0	2
4310.0	2
7200.0	2
4520.0	2
4350.0	2
4450.0	2
4830.0	2
4610.0	2
4815.0	2
5400.0	2
2600.0	1
4085.0	1
8400.0	1
7600.0	1
4115.0	1
2300.0	1
3315.0	1
11000.0	1
5700.0	1
4460.0	1
3055.0	1
5600.0	1
4805.0	1
2000.0	1
4360.0	1
3350.0	1
4040.0	1
4030.0	1
8200.0	1
3140.0	1
2800.0	1
4575.0	1
4355.0	1
4410.0	1
5003.0	1
4680.0	1

```
4614.0       1
4080.0       1
3885.0       1
5450.0       1
3750.0       1
7500.0       1
7000.0       1
5250.0       1
8300.0       1
8850.0       1
5065.0       1
Name: count, dtype: int64
```

IV. First we check the issue with the `Screen Size` colomn in df dataset, and than make the colomn numeric.

In [373…
```
df['Screen Size'].unique()
```

Out[373…
```
array(['6.1 inches', '6.7 inches', '5.4 inches', '5.8 inches',
       '6.5 inches', '10.9 inches', '10.2 inches', '7.9 inches',
       '11 inches', '12.9 inches', '13 inches', '6.8 inches',
       '6.6 inches', '7.6 inches', '6.4 inches', '6.9 inches',
       '6.3 inches', '5.3 inches', '6.0 inches', '5.5 inches',
       '5.7 inches', '5.2 inches', '14.6 inches', '12.4 inches',
       '8.7 inches', '10.5 inches', '8 inches', '10.1 inches',
       '6.74 inches', '6.72 inches', '7.8 inches', '6.55 inches',
       '6.43 inches', '6.49 inches', '6.52 inches', '6.78 inches',
       '6.59 inches', '6.44 inches', '6.41 inches', '6.01 inches',
       '6.67 inches', '6.28 inches', '11.61 inches', '6.31 inches',
       '6.58 inches', '6.38 inches', '6.56 inches', '5.88 inches',
       '6.22 inches', '5.0 inches', '6.51 inches', '6.35 inches',
       '6.53 inches', '6.39 inches', '6.47 inches', '10.4 inches',
       '12.3 inches', '7.82 inches', '6.83 inches', '11.6 inches',
       '12.1 inches', '6.82 inches', '7.1 inches', '11.5 inches',
       '6.73 inches', '6.36 inches', '6.09 inches',
       '6.7 inches (main), 2.7 inches (external)',
       '6.9 inches (internal), 4.0 inches (external)',
       '6.7 inches (internal), 3.6 inches (external)',
       '6.9 inches (unfolded)', '8.0 inches (unfolded)', '6.57 inches',
       '7.8 inches (unfolded)', '7.85 inches (unfolded)', '7.93 inches',
       '7.92 inches', '12.2 inches', '13.2 inches', '5.6 inches',
       '6.2 inches', '6.34 inches', '6.71 inches', '7.85 inches',
       '9.7 inches', '11.0 inches', '6.95 inches', '6.85 inches',
       '6.63 inches', '7.09 inches', '6.81 inches', '6.76 inches',
       '12.0 inches', '12.6 inches', '13.0 inches', '13.5 inches',
       '6.79 inches'], dtype=object)
```

In [374…
```
# I have to see the all the value_counts of the Screen Size colomn, not the ... as
pd.set_option('display.max_rows', None)
df['Screen Size'].value_counts()
```

Out[374…

Screen Size	
6.7 inches	124
6.5 inches	76
6.67 inches	69
6.6 inches	64
6.78 inches	61
6.1 inches	51
6.8 inches	39
6.55 inches	22
6.43 inches	21
6.44 inches	20
6.82 inches	16
11 inches	15
6.52 inches	15
6.4 inches	15
6.9 inches	14
6.81 inches	14
6.72 inches	14
6.56 inches	12
6.58 inches	12
6.74 inches	12
6.3 inches	11
7.6 inches	10
10.4 inches	10
6.59 inches	9
12.1 inches	9
5.8 inches	9
6.53 inches	7
10.1 inches	7
6.39 inches	7
5.5 inches	7
6.0 inches	6
6.73 inches	6
5.4 inches	6
9.7 inches	5
13 inches	5
6.57 inches	5
6.28 inches	5
6.51 inches	4
6.36 inches	4
6.49 inches	4
7.92 inches	4
10.9 inches	4
8.7 inches	4
12.9 inches	4
7.9 inches	4
12.4 inches	3
11.5 inches	3
8 inches	3
10.5 inches	3
6.38 inches	3
11.0 inches	3
6.76 inches	3
12.3 inches	3
6.22 inches	3
11.6 inches	3

```
6.83 inches                                          3
10.2 inches                                          2
13.0 inches                                          2
6.85 inches                                          2
7.09 inches                                          2
5.7 inches                                           2
14.6 inches                                          2
6.47 inches                                          2
6.41 inches                                          2
11.61 inches                                         2
13.2 inches                                          2
6.2 inches                                           2
6.35 inches                                          2
12.6 inches                                          2
6.7 inches (main), 2.7 inches (external)             2
7.8 inches                                           1
5.3 inches                                           1
5.2 inches                                           1
7.1 inches                                           1
7.82 inches                                          1
5.0 inches                                           1
5.88 inches                                          1
6.31 inches                                          1
6.01 inches                                          1
6.9 inches (internal), 4.0 inches (external)         1
6.09 inches                                          1
5.6 inches                                           1
12.2 inches                                          1
7.85 inches (unfolded)                               1
7.93 inches                                          1
7.8 inches (unfolded)                                1
6.7 inches (internal), 3.6 inches (external)         1
6.9 inches (unfolded)                                1
8.0 inches (unfolded)                                1
6.63 inches                                          1
6.95 inches                                          1
6.71 inches                                          1
7.85 inches                                          1
6.34 inches                                          1
12.0 inches                                          1
13.5 inches                                          1
6.79 inches                                          1
Name: count, dtype: int64
```

In [375…

```python
# Function to clean the Screen Size values
def clean_screen_size(value):
    import re

    # Remove unwanted text and extract numbers
    numbers = re.findall(r'\d+\.?\d*', value)  # Extract all numeric values

    # Convert to float
    numbers = [float(num) for num in numbers]

    if numbers:
        return max(numbers)  # Take the largest value (assuming it's the main scree
```

```
    return None  # Return None if no valid number is found

# Apply cleaning function
df['Screen Size'] = df['Screen Size'].apply(clean_screen_size)
```

In [376…

```
df['Screen Size'].value_counts()
```

Out[376…

Screen Size	
6.70	127
6.50	76
6.67	69
6.60	64
6.78	61
6.10	51
6.80	39
6.55	22
6.43	21
6.44	20
11.00	18
6.82	16
6.90	16
6.52	15
6.40	15
6.72	14
6.81	14
6.74	12
6.56	12
6.58	12
6.30	11
7.60	10
10.40	10
5.80	9
6.59	9
12.10	9
6.39	7
5.50	7
13.00	7
6.53	7
10.10	7
6.73	6
6.00	6
5.40	6
6.57	5
9.70	5
6.28	5
10.90	4
7.90	4
12.90	4
6.36	4
7.92	4
6.49	4
6.51	4
8.70	4
8.00	4
6.22	3
12.40	3
6.76	3
11.50	3
12.30	3
6.83	3
10.50	3
6.38	3
11.60	3

```
10.20      2
7.80       2
7.09       2
5.70       2
14.60      2
6.85       2
11.61      2
6.41       2
6.47       2
6.35       2
13.20      2
6.20       2
12.60      2
7.85       2
5.30       1
5.20       1
6.01       1
5.00       1
5.88       1
6.31       1
12.20      1
7.82       1
7.93       1
6.09       1
7.10       1
6.95       1
6.71       1
5.60       1
6.34       1
6.63       1
12.00      1
13.50      1
6.79       1
Name: count, dtype: int64
```

In [377…
```
# Now rename the Screen Size colomn to Screen Size(inches)
df.rename(columns={'Screen Size':'Screen Size(inches)'},inplace=True)
```

V. First we check the issue with the `Launched Price (Pakistan,India,China,USA,Dubai)` colomn in df dataset, and than make the colomn numeric.

1. Pakistan

In [378…
```
df['Launched Price (Pakistan)'].unique()
```

Out[378…

```
array(['PKR 224,999', 'PKR 234,999', 'PKR 244,999', 'PKR 249,999',
       'PKR 259,999', 'PKR 274,999', 'PKR 284,999', 'PKR 294,999',
       'PKR 314,999', 'PKR 324,999', 'PKR 344,999', 'PKR 204,999',
       'PKR 214,999', 'PKR 264,999', 'PKR 304,999', 'PKR 184,999',
       'PKR 194,999', 'PKR 364,999', 'PKR 174,999', 'PKR 354,999',
       'PKR 384,999', 'PKR 334,999', 'PKR 159,999', 'PKR 169,999',
       'PKR 179,999', 'PKR 219,999', 'PKR 239,999', 'PKR 269,999',
       'PKR 289,999', 'PKR 309,999', 'PKR 164,999', 'PKR 149,999',
       'PKR 79,999', 'PKR 89,999', 'PKR 49,999', 'PKR 59,999',
       'PKR 69,999', 'PKR 189,999', 'PKR 279,999', 'PKR 359,999',
       'PKR 399,999', 'PKR 450,000', 'PKR 480,000', 'PKR 400,000',
       'PKR 430,000', 'PKR 350,000', 'PKR 380,000', 'PKR 460,000',
       'PKR 360,000', 'PKR 390,000', 'PKR 320,000', 'PKR 420,000',
       'PKR 300,000', 'PKR 330,000', 'PKR 500,000', 'PKR 550,000',
       'PKR 530,000', 'PKR 85,000', 'PKR 95,000', 'PKR 75,000',
       'PKR 70,000', 'PKR 80,000', 'PKR 60,000', 'PKR 50,000',
       'PKR 105,000', 'PKR 180,000', 'PKR 200,000', 'PKR 150,000',
       'PKR 170,000', 'PKR 160,000', 'PKR 140,000', 'PKR 120,000',
       'PKR 45,000', 'PKR 40,000', 'PKR 25,000', 'PKR 65,000',
       'PKR 280,000', 'PKR 260,000', 'PKR 230,000', 'PKR 250,000',
       'PKR 100,000', 'PKR 68,000', 'PKR 35,000', 'PKR 28,000',
       'PKR 119,999', 'PKR 55,000', 'PKR 299,999', 'PKR 199,999',
       'PKR 39,999', 'PKR 29,999', 'PKR 139,999', 'PKR 64,999',
       'PKR 54,999', 'PKR 109,999', 'PKR 99,999', 'PKR 94,999',
       'PKR 229,999', 'PKR 104,999', 'PKR 34,999', 'PKR 42,999',
       'PKR 22,999', 'PKR 24,999', 'PKR 19,999', 'PKR 27,999',
       'PKR 18,999', 'PKR 44,999', 'PKR 18,499', 'PKR 22,499',
       'PKR 84,999', 'PKR 85,999', 'PKR 74,999', 'PKR 26,999',
       'PKR 58,999', 'PKR 62,999', 'PKR 129,999', 'PKR 32,999',
       'PKR 25,999', 'PKR 23,999', 'PKR 21,999', 'PKR 37,999',
       'PKR 47,999', 'PKR 52,999', 'PKR 36,999', 'PKR 124,999',
       'PKR 57,999', 'PKR 61,999', 'PKR 56,999', 'PKR 209,999',
       'PKR 349,999', 'PKR 389,999', 'PKR 134,999', 'PKR 48,999',
       'PKR 46,999', 'PKR 38,999', 'PKR 429,999', 'PKR 319,999',
       'PKR 449,999', 'PKR 339,999', 'PKR 469,999', 'PKR 52,000',
       'PKR 161,500', 'PKR 197,000', 'PKR 55,999', 'PKR 45,999',
       'PKR 72,999', 'PKR 35,999', 'PKR 28,999', 'PKR 30,999',
       'PKR 41,999', 'PKR 31,999', 'PKR 15,999', 'PKR 369,999',
       'PKR 66,220', 'PKR 71,220', 'PKR 604,999', 'PKR 544,999',
       'Not available'], dtype=object)
```

In [379…

```
df['Launched Price (Pakistan)'].value_counts()
```

Out[379…

Launched Price (Pakistan)	
PKR 79,999	39
PKR 89,999	38
PKR 69,999	38
PKR 59,999	35
PKR 54,999	32
PKR 39,999	31
PKR 49,999	27
PKR 64,999	27
PKR 34,999	27
PKR 109,999	26
PKR 99,999	25
PKR 29,999	24
PKR 169,999	22
PKR 199,999	22
PKR 44,999	21
PKR 74,999	21
PKR 119,999	20
PKR 179,999	20
PKR 219,999	17
PKR 159,999	16
PKR 129,999	15
PKR 149,999	15
PKR 189,999	13
PKR 94,999	13
PKR 84,999	12
PKR 139,999	12
PKR 24,999	10
PKR 249,999	9
PKR 19,999	7
PKR 22,999	7
PKR 27,999	7
PKR 299,999	7
PKR 32,999	7
PKR 70,000	6
PKR 42,999	6
PKR 229,999	6
PKR 259,999	6
PKR 350,000	6
PKR 349,999	5
PKR 209,999	5
PKR 60,000	5
PKR 194,999	5
PKR 204,999	5
PKR 324,999	5
PKR 344,999	5
PKR 224,999	5
PKR 314,999	5
PKR 184,999	4
PKR 50,000	4
PKR 40,000	4
PKR 37,999	4
PKR 21,999	4
PKR 85,000	4
PKR 25,999	4
PKR 239,999	4

PKR 269,999	4
PKR 28,999	3
PKR 319,999	3
PKR 294,999	3
PKR 48,999	3
PKR 389,999	3
PKR 75,000	3
PKR 320,000	3
PKR 380,000	3
PKR 95,000	3
PKR 399,999	3
PKR 80,000	3
PKR 180,000	3
PKR 150,000	3
PKR 234,999	2
PKR 284,999	2
PKR 274,999	2
PKR 244,999	2
PKR 214,999	2
PKR 174,999	2
PKR 384,999	2
PKR 364,999	2
PKR 30,999	2
PKR 35,999	2
PKR 304,999	2
PKR 289,999	2
PKR 354,999	2
PKR 23,999	2
PKR 45,999	2
PKR 47,999	2
PKR 280,000	2
PKR 309,999	2
PKR 120,000	2
PKR 200,000	2
PKR 480,000	2
PKR 359,999	2
PKR 450,000	2
PKR 430,000	2
PKR 52,999	2
PKR 124,999	2
PKR 38,999	2
PKR 18,999	2
PKR 26,999	2
PKR 62,999	2
PKR 334,999	1
PKR 264,999	1
PKR 45,000	1
PKR 105,000	1
PKR 170,000	1
PKR 230,000	1
PKR 100,000	1
PKR 68,000	1
PKR 35,000	1
PKR 28,000	1
PKR 300,000	1
PKR 420,000	1

```
PKR 390,000       1
PKR 360,000       1
PKR 400,000       1
PKR 279,999       1
PKR 164,999       1
PKR 460,000       1
PKR 530,000       1
PKR 550,000       1
PKR 500,000       1
PKR 160,000       1
PKR 25,000        1
PKR 140,000       1
PKR 65,000        1
PKR 330,000       1
PKR 55,000        1
PKR 260,000       1
PKR 250,000       1
PKR 58,999        1
PKR 18,499        1
PKR 104,999       1
PKR 22,499        1
PKR 85,999        1
PKR 57,999        1
PKR 36,999        1
PKR 429,999       1
PKR 449,999       1
PKR 46,999        1
PKR 134,999       1
PKR 61,999        1
PKR 56,999        1
PKR 161,500       1
PKR 52,000        1
PKR 469,999       1
PKR 339,999       1
PKR 72,999        1
PKR 55,999        1
PKR 197,000       1
PKR 41,999        1
PKR 31,999        1
PKR 15,999        1
PKR 369,999       1
PKR 66,220        1
PKR 71,220        1
PKR 604,999       1
PKR 544,999       1
Not available     1
Name: count, dtype: int64
```

In [380…

```
# Now we have to remove the Rs. from the Launched Price (Pakistan) colomn along wit
# numeric.
df['Launched Price (Pakistan)'] = df['Launched Price (Pakistan)'].str.replace('PKR'
df['Launched Price (Pakistan)'] = df['Launched Price (Pakistan)'].replace('Not avai
```

In [381…

```
# rename the Launched Price (Pakistan) colomn to Launched Price(PKR)
df.rename(columns={'Launched Price (Pakistan)': 'Launched Price(PKR)'}, inplace=Tru
```

2. India

In [382…
```
df['Launched Price (India)'].unique()
```

Out[382…
```
array(['INR 79,999', 'INR 84,999', 'INR 89,999', 'INR 94,999',
       'INR 104,999', 'INR 99,999', 'INR 114,999', 'INR 109,999',
       'INR 124,999', 'INR 74,999', 'INR 119,999', 'INR 134,999',
       'INR 69,999', 'INR 144,999', 'INR 69,900', 'INR 74,900',
       'INR 84,900', 'INR 94,900', 'INR 119,900', 'INR 129,900',
       'INR 139,900', 'INR 149,900', 'INR 64,900', 'INR 99,900',
       'INR 109,900', 'INR 89,900', 'INR 54,900', 'INR 29,900',
       'INR 39,900', 'INR 49,900', 'INR 71,900', 'INR 159,900',
       'INR 179,900', 'INR 199,900', 'INR 104,900', 'INR 114,900',
       'INR 79,900', 'INR 154,900', 'INR 174,900', 'INR 134,900',
       'INR 27,999', 'INR 32,999', 'INR 22,999', 'INR 19,999',
       'INR 24,999', 'INR 18,499', 'INR 21,999', 'INR 14,999',
       'INR 23,999', 'INR 26,999', 'INR 17,999', 'INR 20,999',
       'INR 12,999', 'INR 15,999', 'INR 18,999', 'INR 1,04,999',
       'INR 77,999', 'INR 64,999', 'INR 59,999', 'INR 28,999',
       'INR 17,499', 'INR 18,990', 'INR 9,990', 'INR 32,990',
       'INR 22,990', 'INR 19,990', 'INR 1,14,999', 'INR 85,999',
       'INR 49,999', 'INR 1,09,999', 'INR 16,999', 'INR 11,999',
       'INR 54,999', 'INR 44,999', 'INR 9,999', 'INR 8,499', 'INR 34,999',
       'INR 39,999', 'INR 45,999', 'INR 42,999', 'INR 41,999',
       'INR 38,999', 'INR 50,999', 'INR 37,999', 'INR 36,999',
       'INR 58,999', 'INR 129,999', 'INR 139,999', 'INR 149,999',
       'INR 29,999', 'INR 13,999', 'INR 10,999', 'INR 27,990',
       'INR 31,990', 'INR 33,990', 'INR 10,990', 'INR 13,990',
       'INR 24,990', 'INR 13,490', 'INR 26,990', 'INR 35,990',
       'INR 39,990', 'INR 14,990', 'INR 11,990', 'INR 21,990',
       'INR 25,999', 'INR 47,999', 'INR 31,999', 'INR 43,999',
       'INR 159,999', 'INR 52,999', 'INR 46,999', 'INR 19,000',
       'INR 8,999', 'INR 18,000', 'INR 20,000', 'INR 16,990',
       'INR 65,999', 'INR 28,756', 'INR 30,999', 'INR 19,499',
       'INR 13,499', 'INR 53,999', 'INR 33,999', 'INR 35,999',
       'INR 7,499', 'INR 21,400', 'INR 249,999', 'INR 199,999',
       'INR 259,999', 'INR 274,999', 'INR 67,999', 'INR 12,499',
       'INR 9,499', 'INR 49,990', 'INR 58,590', 'INR 179,999',
       'INR 6,999', 'INR 169,999', 'INR 11,499', 'INR 10,499',
       'INR 7,999', 'INR 5,999', 'INR 72,999', 'INR 164,999',
       'INR 176,999', 'INR 200,999'], dtype=object)
```

In [383…
```
df['Launched Price (India)'].value_counts()
```

Out[383…

Launched Price (India)	
INR 29,999	35
INR 39,999	31
INR 22,999	30
INR 44,999	29
INR 34,999	27
INR 14,999	27
INR 24,999	26
INR 19,999	24
INR 54,999	23
INR 49,999	22
INR 59,999	19
INR 17,999	19
INR 13,999	18
INR 21,999	18
INR 12,999	18
INR 16,999	18
INR 74,999	18
INR 99,999	17
INR 27,999	17
INR 84,999	16
INR 15,999	16
INR 64,999	15
INR 10,999	15
INR 79,999	15
INR 32,999	14
INR 89,999	14
INR 69,999	14
INR 18,999	13
INR 11,999	13
INR 109,999	12
INR 26,999	12
INR 119,999	11
INR 94,999	10
INR 37,999	9
INR 9,999	9
INR 25,999	8
INR 119,900	8
INR 36,999	8
INR 129,999	8
INR 84,900	7
INR 12,499	7
INR 42,999	7
INR 109,900	7
INR 89,900	6
INR 20,999	6
INR 129,900	6
INR 94,900	6
INR 74,900	6
INR 104,999	5
INR 99,900	5
INR 18,990	5
INR 28,999	5
INR 139,900	5
INR 79,900	5
INR 7,499	5

INR 11,499	5
INR 23,999	5
INR 114,999	4
INR 124,999	4
INR 149,999	4
INR 47,999	4
INR 69,900	4
INR 41,999	4
INR 31,999	4
INR 134,999	3
INR 46,999	3
INR 38,999	3
INR 249,999	3
INR 159,999	3
INR 30,999	3
INR 45,999	3
INR 22,990	3
INR 64,900	3
INR 104,900	3
INR 9,499	3
INR 20,000	2
INR 149,900	2
INR 154,900	2
INR 18,499	2
INR 19,990	2
INR 18,000	2
INR 39,900	2
INR 8,499	2
INR 24,990	2
INR 27,990	2
INR 43,999	2
INR 19,499	2
INR 10,499	2
INR 7,999	2
INR 35,999	2
INR 6,999	2
INR 16,990	2
INR 139,999	2
INR 144,999	1
INR 1,09,999	1
INR 50,999	1
INR 85,999	1
INR 114,900	1
INR 174,900	1
INR 199,900	1
INR 49,900	1
INR 54,900	1
INR 77,999	1
INR 9,990	1
INR 17,499	1
INR 32,990	1
INR 1,14,999	1
INR 1,04,999	1
INR 179,900	1
INR 159,900	1
INR 71,900	1

```
INR 29,900       1
INR 134,900      1
INR 13,490       1
INR 13,990       1
INR 8,999        1
INR 19,000       1
INR 52,999       1
INR 35,990       1
INR 14,990       1
INR 39,990       1
INR 21,990       1
INR 11,990       1
INR 26,990       1
INR 10,990       1
INR 58,999       1
INR 31,990       1
INR 33,990       1
INR 13,499       1
INR 53,999       1
INR 28,756       1
INR 274,999      1
INR 259,999      1
INR 199,999      1
INR 21,400       1
INR 33,999       1
INR 65,999       1
INR 49,990       1
INR 67,999       1
INR 169,999      1
INR 179,999      1
INR 58,590       1
INR 5,999        1
INR 72,999       1
INR 164,999      1
INR 176,999      1
INR 200,999      1
Name: count, dtype: int64
```

In [384…

```
# Now we have to remove the Rs. from the Launched Price (India) colomn along with r
# numeric.
df['Launched Price (India)'] = df['Launched Price (India)'].str.replace('INR', '').
df['Launched Price (India)'] = df['Launched Price (India)'].replace('Not available'
```

In [385…

```
# reame the Launched Price (India) colomn to Launched Price(INR)
df = df.rename(columns={'Launched Price (India)': 'Launched Price(INR)'})
```

3. China

In [386…

```
df['Launched Price (China)'].unique()
```

Out[386…

```
array(['CNY 5,799', 'CNY 6,099', 'CNY 6,499', 'CNY 6,199', 'CNY 6,999',
       'CNY 7,099', 'CNY 7,499', 'CNY 7,799', 'CNY 8,199', 'CNY 5,299',
       'CNY 5,599', 'CNY 5,999', 'CNY 5,699', 'CNY 6,799', 'CNY 7,299',
       'CNY 7,999', 'CNY 8,699', 'CNY 5,199', 'CNY 5,499', 'CNY 5,899',
       'CNY 6,299', 'CNY 7,199', 'CNY 8,099', 'CNY 8,499', 'CNY 8,999',
       'CNY 7,699', 'CNY 8,299', 'CNY 8,799', 'CNY 9,199', 'CNY 8,399',
       'CNY 7,399', 'CNY 8,388', 'CNY 8,788', 'CNY 9,188', 'CNY 9,788',
       'CNY 10,388', 'CNY 10,088', 'CNY 10,688', 'CNY 11,288',
       'CNY 3,299', 'CNY 3,699', 'CNY 2,299', 'CNY 2,699', 'CNY 3,499',
       'CNY 3,799', 'CNY 6,399', 'CNY 9,799', 'CNY 10,399', 'CNY 11,199',
       'CNY 6,599', 'CNY 11,999', 'CNY 12,999', 'CNY 10,999', 'CNY 2,199',
       'CNY 2,499', 'CNY 1,799', 'CNY 1,999', 'CNY 1,599', 'CNY 1,499',
       'CNY 1,699', 'CNY 1,199', 'CNY 1,399', 'CNY 2,099', 'CNY 1,099',
       'CNY 1,299', 'CNY 2,399', 'CNY 999', 'CNY 3,099', 'CNY 1,899',
       'CNY 9,999', 'CNY 9,499', 'CNY 4,299', 'CNY 2,999', 'CNY 899',
       'CNY 799', 'CNY 4,999', 'CNY 2,799', 'CNY 4,699', 'CNY 4,199',
       'CNY 3,199', 'CNY 3,899', 'CNY 3,999', 'CNY 4,499', 'CNY 4,599',
       'CNY 3,599', 'CNY 3,399', 'CNY 3,400', 'CNY 3,600', 'CNY 2,800',
       'CNY 2,900', 'CNY 3,000', 'CNY 2,700', 'CNY 2,600', 'CNY 2,400',
       'CNY 2,500', 'CNY 2,000', 'CNY 2,100', 'CNY 1,800', 'CNY 1,900',
       'CNY 1,500', 'CNY 1,600', 'CNY 2,200', 'CNY 1,400', 'CNY 1,700',
       'CNY 1,200', 'CNY 1,100', 'CNY 900', 'CNY 1,300', 'CNY 1,050',
       'CNY 3,200', 'CNY 2,300', 'CNY 1,350', 'CNY 1,550', 'CNY 4,799',
       'CNY 2,899', 'CNY 1,998', 'CNY 4,488', 'CNY 5,988', 'CNY 8,988',
       'CNY 17,999', 'CNY 4,088', 'CNY 14,999', 'CNY 6,988', 'CNY 7,988',
       'CNY 13,999', 'CNY 9,988', 'CNY 13,499', 'CNY 14,499', 'CNY 1,250',
       'CNY 599', 'CNY 549', 'CNY 2,599', 'CNY 699', 'CNY 499', '¥13,999',
       'CNY 15,999', 'CNY 17,999\xa0'], dtype=object)
```

In [387…

```
df['Launched Price (China)'].value_counts()
```

Out[387…

Launched Price (China)	
CNY 2,499	36
CNY 1,499	32
CNY 3,499	31
CNY 2,199	29
CNY 1,799	29
CNY 999	29
CNY 1,199	26
CNY 2,299	25
CNY 2,999	25
CNY 6,999	23
CNY 5,999	22
CNY 1,999	21
CNY 1,299	21
CNY 3,999	21
CNY 1,699	20
CNY 1,599	18
CNY 5,499	17
CNY 3,299	17
CNY 7,999	17
CNY 1,399	16
CNY 1,099	15
CNY 3,199	15
CNY 2,699	14
CNY 6,499	14
CNY 2,799	13
CNY 4,999	13
CNY 2,099	12
CNY 2,399	11
CNY 1,899	11
CNY 7,499	11
CNY 899	11
CNY 4,499	10
CNY 3,699	10
CNY 3,799	9
CNY 4,299	9
CNY 6,299	8
CNY 7,299	8
CNY 6,199	8
CNY 8,199	7
CNY 8,999	7
CNY 4,699	7
CNY 3,599	6
CNY 5,199	6
CNY 1,200	6
CNY 1,600	5
CNY 4,799	5
CNY 9,999	5
CNY 1,500	5
CNY 2,800	5
CNY 5,899	5
CNY 4,199	5
CNY 8,499	5
CNY 8,799	5
CNY 4,599	5
CNY 2,500	4

CNY 5,299	4
CNY 7,199	4
CNY 9,199	4
CNY 1,100	4
CNY 1,300	4
CNY 599	4
CNY 6,799	4
CNY 5,599	4
CNY 3,099	4
CNY 10,999	4
CNY 799	4
CNY 8,988	3
CNY 7,799	3
CNY 5,799	3
CNY 1,900	3
CNY 6,399	3
CNY 6,599	3
CNY 12,999	3
CNY 1,400	3
CNY 2,400	3
CNY 2,899	3
CNY 8,699	3
CNY 2,000	3
CNY 1,800	3
CNY 7,699	2
CNY 7,099	2
CNY 7,988	2
CNY 549	2
CNY 2,599	2
CNY 1,700	2
CNY 2,200	2
CNY 2,900	2
CNY 9,799	2
CNY 11,999	2
CNY 3,600	2
CNY 5,699	2
CNY 8,299	2
CNY 699	2
CNY 3,000	2
CNY 2,700	2
CNY 6,988	2
CNY 13,999	2
CNY 3,899	2
CNY 6,099	1
CNY 8,788	1
CNY 8,388	1
CNY 11,288	1
CNY 10,388	1
CNY 9,188	1
CNY 10,688	1
CNY 9,499	1
CNY 10,399	1
CNY 11,199	1
CNY 9,788	1
CNY 10,088	1
CNY 8,099	1

```
CNY 8,399       1
CNY 7,399       1
CNY 2,600       1
CNY 2,100       1
CNY 3,400       1
CNY 3,399       1
CNY 1,550       1
CNY 3,200       1
CNY 900         1
CNY 1,050       1
CNY 4,088       1
CNY 17,999      1
CNY 5,988       1
CNY 4,488       1
CNY 1,998       1
CNY 2,300       1
CNY 1,350       1
CNY 14,999      1
CNY 1,250       1
CNY 14,499      1
CNY 13,499      1
CNY 9,988       1
CNY 499         1
¥13,999         1
CNY 15,999      1
CNY 17,999      1
Name: count, dtype: int64
```

In [388…

```
# Now we have to remove the Rs. from the Launched Price (China) colomn along with r
# numeric.
df['Launched Price (China)'] = df['Launched Price (China)'].str.replace('CNY', '').
df['Launched Price (China)'] = df['Launched Price (China)'].replace('Not available'
```

In [389…

```
# rename the Launched Price (China) colomn to Launched Price(CNY)
df = df.rename(columns={'Launched Price (China)': 'Launched Price(CNY)'})
```

4. Dubai

In [390…

```
df['Launched Price (Dubai)'].unique()
```

Out[390…

```
array(['AED 2,799', 'AED 2,999', 'AED 3,199', 'AED 3,399', 'AED 3,599',
       'AED 3,499', 'AED 3,699', 'AED 3,899', 'AED 3,799', 'AED 3,999',
       'AED 4,199', 'AED 2,699', 'AED 4,399', 'AED 4,499', 'AED 2,599',
       'AED 3,299', 'AED 4,599', 'AED 2,499', 'AED 4,999', 'AED 4,299',
       'AED 4,799', 'AED 5,099', 'AED 3,099', 'AED 2,099', 'AED 1,199',
       'AED 1,499', 'AED 1,799', 'AED 5,299', 'AED 5,799', 'AED 6,099',
       'AED 4,899', 'AED 6,999', 'AED 7,499', 'AED 4,099', 'AED 6,499',
       'AED 7,099', 'AED 1,399', 'AED 1,299', 'AED 1,099', 'AED 899',
       'AED 999', 'AED 1,699', 'AED 799', 'AED 699', 'AED 499',
       'AED 5,499', 'AED 1,999', 'AED 599', 'AED 2,199', 'AED 1,899',
       'AED 399', 'AED 2,899', 'AED 2,299', 'AED 2,399', 'AED 4,699',
       'AED 1,649', 'AED 549', 'AED 649', 'AED 1,599', 'AED 749',
       'AED 5,199', 'AED 1,000', 'AED 1,200', 'AED 1,300', 'AED 1,100',
       'AED 950', 'AED 900', 'AED 850', 'AED 800', 'AED 750', 'AED 700',
       'AED 1,500', 'AED 1,700', 'AED 1,400', 'AED 1,725', 'AED 1,840',
       'AED 1,460', 'AED 1,520', 'AED 1,330', 'AED 1,250', 'AED 1,320',
       'AED 1,150', 'AED 1,210', 'AED 970', 'AED 1,030', 'AED 860',
       'AED 1,070', 'AED 1,140', 'AED 1,390', 'AED 960', 'AED 1,280',
       'AED 720', 'AED 780', 'AED 1,020', 'AED 920', 'AED 660', 'AED 600',
       'AED 830', 'AED 500', 'AED 620', 'AED 680', 'AED 580', 'AED 820',
       'AED 880', 'AED 1,620', 'AED 1,800', 'AED 740', 'AED 1,050',
       'AED 770', 'AED 730', 'AED 849', 'AED 949', 'AED 1,049',
       'AED 1,175', 'AED 1,350', 'AED 1,275', 'AED 1,675', 'AED 925',
       'AED 1,850', 'AED 550', 'AED 650', 'AED 9,999', 'AED 8,999',
       'AED 10,499', 'AED 11,099', 'AED 629', 'AED 619', 'AED 449',
       'AED 5,999', 'AED 6,199', 'AED 870', 'AED 349', 'AED 299',
       'AED 1,149', 'AED 1,249', 'AED 2,049', 'AED 6,599', 'AED 1,029',
       'AED 7,199', 'AED 7,699', 'AED 8,699'], dtype=object)
```

In [391…

```
df['Launched Price (Dubai)'].value_counts()
```

Out[391…

Launched Price (Dubai)	
AED 1,499	39
AED 1,099	35
AED 1,299	29
AED 3,299	25
AED 1,699	25
AED 999	24
AED 899	23
AED 699	23
AED 1,899	23
AED 1,199	22
AED 1,999	21
AED 2,199	21
AED 2,999	20
AED 799	20
AED 4,199	20
AED 3,699	20
AED 3,999	19
AED 2,499	18
AED 599	18
AED 1,799	17
AED 3,599	16
AED 4,399	16
AED 1,399	15
AED 749	15
AED 2,599	14
AED 3,199	14
AED 2,799	13
AED 2,099	12
AED 1,000	12
AED 3,799	11
AED 1,599	11
AED 849	10
AED 1,200	10
AED 549	10
AED 499	10
AED 2,299	10
AED 649	9
AED 1,400	8
AED 4,799	8
AED 1,500	8
AED 3,499	8
AED 4,599	8
AED 4,099	7
AED 3,399	7
AED 4,499	7
AED 2,699	6
AED 3,899	6
AED 399	6
AED 2,399	6
AED 1,100	6
AED 5,499	5
AED 4,699	5
AED 1,300	5
AED 6,999	5
AED 5,199	4

AED 4,999	4
AED 2,899	4
AED 660	4
AED 600	4
AED 3,099	3
AED 4,299	3
AED 620	3
AED 1,675	3
AED 1,275	3
AED 1,175	3
AED 780	3
AED 800	3
AED 9,999	3
AED 349	3
AED 900	3
AED 850	3
AED 700	2
AED 1,700	2
AED 5,799	2
AED 4,899	2
AED 750	2
AED 5,099	2
AED 949	2
AED 1,350	2
AED 2,049	2
AED 1,149	2
AED 1,049	2
AED 6,199	2
AED 925	2
AED 1,850	2
AED 1,150	2
AED 770	2
AED 680	2
AED 960	2
AED 970	2
AED 1,320	2
AED 1,210	2
AED 1,250	2
AED 860	2
AED 1,840	1
AED 1,460	1
AED 950	1
AED 1,649	1
AED 6,499	1
AED 6,099	1
AED 7,499	1
AED 7,099	1
AED 5,299	1
AED 1,725	1
AED 1,280	1
AED 720	1
AED 1,030	1
AED 1,330	1
AED 1,520	1
AED 1,050	1
AED 740	1

```
AED 1,800      1
AED 1,620      1
AED 820        1
AED 880        1
AED 580        1
AED 830        1
AED 920        1
AED 500        1
AED 1,020      1
AED 1,070      1
AED 1,390      1
AED 1,140      1
AED 730        1
AED 550        1
AED 11,099     1
AED 10,499     1
AED 8,999      1
AED 650        1
AED 5,999      1
AED 449        1
AED 629        1
AED 619        1
AED 299        1
AED 870        1
AED 1,249      1
AED 6,599      1
AED 1,029      1
AED 7,199      1
AED 7,699      1
AED 8,699      1
Name: count, dtype: int64
```

In [392…

```
# Now we have to remove the AED from the Launched Price (Dubai) colomn along with r
# numeric.
df['Launched Price (Dubai)'] = df['Launched Price (Dubai)'].str.replace('AED', '').
df['Launched Price (Dubai)'] = df['Launched Price (Dubai)'].replace('Not available'
```

In [393…

```
# rename the Launched Price (Dubai) colomn to Launched Price(AED)
df = df.rename(columns={'Launched Price (Dubai)': 'Launched Price(AED)'})
```

5. USA

In [394…

```
df['Launched Price (USA)'].unique()
```

Out[394…

```
array(['USD 799', 'USD 849', 'USD 899', 'USD 949', 'USD 999', 'USD 1,049',
       'USD 1,099', 'USD 1,199', 'USD 1,299', 'USD 1,399', 'USD 699',
       'USD 1,249', 'USD 749', 'USD 599', 'USD 329', 'USD 429', 'USD 399',
       'USD 499', 'USD 1,599', 'USD 1,799', 'USD 1,899', 'USD 449',
       'USD 349', 'USD 299', 'USD 249', 'USD 199', 'USD 179', 'USD 169',
       'USD 129', 'USD 1,499', 'USD 549', 'USD 229', 'USD 149', 'USD 649',
       'USD 99', 'USD 219', 'USD 139', 'USD 159', 'USD 269', 'USD 319',
       'USD 239', 'USD 279', 'USD 189', 'USD 259', 'USD 470', 'USD 500',
       'USD 380', 'USD 400', 'USD 420', 'USD 360', 'USD 340', 'USD 320',
       'USD 270', 'USD 290', 'USD 250', 'USD 220', 'USD 240', 'USD 300',
       'USD 200', 'USD 260', 'USD 180', 'USD 160', 'USD 210', 'USD 230',
       'USD 130', 'USD 170', 'USD 190', 'USD 150', 'USD 440', 'USD 330',
       'USD 350', 'USD 280', 'USD 310', 'USD 634.99', 'USD 790.77',
       'USD 374.90', 'USD 399.00', 'USD 429.00', 'USD 349.00',
       'USD 379.00', 'USD 299.00', 'USD 329.00', 'USD 279.00',
       'USD 309.00', 'USD 249.00', 'USD 199.00', 'USD 229.00', 'USD 479',
       'USD 370', 'USD 450', 'USD 2,699', 'USD 2,499', 'USD 2,599',
       'USD 2,799', 'USD 529', 'USD 729', 'USD 119', 'USD 1,699',
       'USD 396,22', 'USD 877', 'USD 89', 'USD 289', 'USD 379', 'USD 109',
       'USD 79', 'USD 1719', 'USD 2,259'], dtype=object)
```

In [395…

```
df['Launched Price (USA)'].value_counts()
```

Out[395…

Launched Price (USA)	
USD 499	44
USD 1,099	43
USD 899	43
USD 299	43
USD 399	42
USD 999	40
USD 199	39
USD 799	38
USD 349	37
USD 599	35
USD 699	32
USD 449	27
USD 1,199	27
USD 249	26
USD 549	23
USD 1,299	22
USD 179	22
USD 229	22
USD 149	16
USD 649	14
USD 279	14
USD 219	12
USD 1,399	11
USD 159	11
USD 749	10
USD 329	9
USD 189	8
USD 169	8
USD 1,799	7
USD 139	7
USD 849	6
USD 1,499	6
USD 949	6
USD 320	6
USD 300	6
USD 379	6
USD 319	5
USD 129	5
USD 350	5
USD 270	5
USD 269	5
USD 99	5
USD 1,899	5
USD 180	5
USD 220	4
USD 239	4
USD 429	4
USD 1,049	4
USD 210	4
USD 250	4
USD 500	4
USD 160	4
USD 380	4
USD 360	4
USD 200	4

```
USD 190        3
USD 170        3
USD 240        3
USD 400        3
USD 450        3
USD 2,499      3
USD 230        3
USD 340        3
USD 370        2
USD 1,599      2
USD 280        2
USD 290        2
USD 279.00     2
USD 479        2
USD 89         2
USD 1,699      2
USD 109        2
USD 150        2
USD 1,249      1
USD 420        1
USD 790.77     1
USD 634.99     1
USD 310        1
USD 330        1
USD 440        1
USD 130        1
USD 260        1
USD 259        1
USD 470        1
USD 329.00     1
USD 379.00     1
USD 299.00     1
USD 349.00     1
USD 374.90     1
USD 429.00     1
USD 399.00     1
USD 2,699      1
USD 249.00     1
USD 199.00     1
USD 229.00     1
USD 309.00     1
USD 529        1
USD 2,799      1
USD 2,599      1
USD 396,22     1
USD 119        1
USD 729        1
USD 877        1
USD 289        1
USD 79         1
USD 1719       1
USD 2,259      1
Name: count, dtype: int64
```

In [396…

```
# Now we have to remove the USD from the Launched Price (USA) colomn along with rem
# numeric.
```

```
df['Launched Price (USA)'] = df['Launched Price (USA)'].str.replace('USD', '').str.
df['Launched Price (USA)'] = df['Launched Price (USA)'].replace('Not available', np
```

In [397…
```
# rename the Launched Price (USA) colomn to Launched Price(USD)
df = df.rename(columns={'Launched Price (USA)': 'Launched Price(USD)'})
```

In [398…
```
df.head()
```

Out[398…

	Company Name	Model Name	Mobile Weight(g)	RAM(GB)	Front Camera	Back Camera	Processor	Battery Capacity(mAh)	Si
0	Apple	iPhone 16 128GB	174.0	6.0	12MP	48MP	A17 Bionic	3600.0	
1	Apple	iPhone 16 256GB	174.0	6.0	12MP	48MP	A17 Bionic	3600.0	
2	Apple	iPhone 16 512GB	174.0	6.0	12MP	48MP	A17 Bionic	3600.0	
3	Apple	iPhone 16 Plus 128GB	203.0	6.0	12MP	48MP	A17 Bionic	4200.0	
4	Apple	iPhone 16 Plus 256GB	203.0	6.0	12MP	48MP	A17 Bionic	4200.0	

VI. First we check the issue with the `Front and Back Camera` colomn in df dataset, and than make the colomn numeric.

I. Front Camera

In [399…
```
df['Front Camera'].unique()
```

Out[399…
```
array(['12MP', '12MP / 4K', '7MP', '10MP', '32MP', '13MP', '5MP', '16MP',
       '8MP', '12MP + 12MP', '2MP', '44MP', '24MP', '20MP+8MP', '20MP',
       '50MP', '25MP', '60MP', '10.7MP', 'Dual 32MP', 'Dual 60MP',
       '60MP (ultrawide) + 8MP (telephoto)', '60MP + 8MP', '11.1MP',
       '10.8MP', '10.5MP', '48MP', '42MP', '10MP, 4MP (UDC)'],
      dtype=object)
```

In [400…
```
df['Front Camera'].value_counts()
```

Out[400…

```
Front Camera
16MP                                    211
32MP                                    207
8MP                                     165
12MP                                     81
5MP                                      47
13MP                                     43
12MP / 4K                                36
50MP                                     30
7MP                                      24
10MP                                     22
2MP                                      12
20MP                                     12
10.8MP                                    7
60MP                                      5
10.5MP                                    3
44MP                                      3
10MP, 4MP (UDC)                           3
60MP + 8MP                                3
12MP + 12MP                               2
60MP (ultrawide) + 8MP (telephoto)        2
20MP+8MP                                  2
10.7MP                                    2
25MP                                      2
24MP                                      1
Dual 32MP                                 1
Dual 60MP                                 1
11.1MP                                    1
48MP                                      1
42MP                                      1
Name: count, dtype: int64
```

In [401…

```python
import re

# Function to clean Front Camera values
def clean_front_camera(value):

    # Extract the numeric value from the string
    numbers = re.findall(r'\d+\.?\d*', value)  # Extract all numeric values

    # Convert to float
    numbers = [float(num) for num in numbers]

    if numbers:
        return max(numbers)  # Take the largest value (assuming it's the main camer

    return None  # Return None if no valid number is found

# Apply cleaning function
df['Front Camera'] = df['Front Camera'].apply(clean_front_camera)
```

In [402…

```python
df['Front Camera'].value_counts()
```

Out[402…

```
Front Camera
16.0    211
32.0    208
8.0     165
12.0    119
5.0      47
13.0     43
50.0     30
10.0     25
7.0      24
20.0     14
2.0      12
60.0     11
10.8      7
10.5      3
44.0      3
10.7      2
25.0      2
24.0      1
11.1      1
48.0      1
42.0      1
Name: count, dtype: int64
```

II. Back Camera

In [403…

```
df['Back Camera'].unique()
```

Out[403…

```
array(['48MP', '50MP + 12MP', '48MP + 12MP', '12MP + 12MP', '12MP',
       '12MP + 12MP + 12MP', '8MP', '12MP + 10MP', '200MP + 12MP',
       '108MP + 12MP', '48MP + 8MP', '50MP + 5MP', '50MP + 2MP',
       '108MP + 8MP', '50MP + 8MP', '13MP + 2MP', '12MP + 16MP', '50MP',
       '16MP', '16MP + 5MP', '13MP', '13MP + 5MP', '16MP + 8MP',
       '13MP + 8MP', '13MP + 6MP', '5MP', '50MP + 48MP', '108MP',
       '64MP + 2MP', '48MP + 48MP', '48MP + 50MP', '48MP + 16MP',
       '48MP + 5MP', '64MP + 8MP', '20MP + 16MP', '16MP + 20MP',
       '50MP + 16MP', '200MP', '64MP', '13MP+2MP', '48MP + 64MP + 48MP',
       '50MP + 32MP + 48MP', '50MP + 50MP + 50MP', '50MP + 50MP',
       '50MP + 8MP + 2MP', '50MP + 50MP + 8MP', '50MP + 32MP + 8MP',
       '64MP + 8MP + 2MP', '8MP + 2MP', '50MP + 50MP + 64MP',
       '50MP + 50MP + 13MP', '50MP + 48MP + 32MP', '64MP + 32MP + 8MP',
       '50MP + 64MP + 8MP', '64MP + 8MP + 2MP + 2MP',
       '50MP + 13MP + 16MP + 2MP', '50MP + 16MP + 13MP + 2MP',
       '48MP + 8MP + 2MP + 2MP', '48MP + 13MP + 12MP',
       '48MP + 13MP + 8MP + 2MP', '13MP + 2MP + 2MP', '16MP + 2MP + 2MP',
       '12MP + 8MP + 2MP + 2MP', '108MP + 2MP', '64MP + 2MP + 2MP',
       '50MP + 8MP + 50MP', '48MP + 2MP + 2MP',
       '50MP (Main) + 50MP (Ultra-wide) + 50MP (Telephoto)',
       '50MP (Main) + 50MP (Ultra-wide)',
       '200MP (Main) + 8MP (Ultra-wide) + 2MP (Macro)',
       '108MP (Main) + 8MP (Ultra-wide) + 2MP (Macro)',
       '50MP (Main) + 8MP (Ultra-wide) + 2MP (Macro)',
       '50MP (Main) + 2MP (Depth)', '40MP',
       '48MP (wide) + 13MP (ultrawide) + 48MP (telephoto)',
       '48MP (wide) + 40MP (ultrawide) + 48MP (telephoto)',
       '50MP (wide) + 13MP (ultrawide) + 12MP (periscope telephoto)',
       '50MP (wide) + 12MP (ultrawide) + 12MP (telephoto)',
       '50MP (wide) + 40MP (ultrawide) + 48MP (telephoto)',
       '50MP (wide) + 48MP (ultrawide) + 48MP (telephoto)',
       '50MP (wide) + 8MP (ultrawide)', '50MP + 13MP + 12MP',
       '50MP + 12.5MP + 48MP', '50MP + 40MP + 50MP', '50MP + 12MP + 40MP',
       '50MP + 12MP + 48MP', '13MP (f/1.8, AF)', '12.2MP', '54MP',
       '160MP', '100MP'], dtype=object)
```

In [404…

```python
import re

# Function to clean Back Camera values
def clean_back_camera(value):

    # Extract the numeric values from the string
    numbers = re.findall(r'\d+\.?\d*', value)  # Extract all numeric values

    # Convert to float
    numbers = [float(num) for num in numbers]

    if numbers:
        return max(numbers)  # Take the largest value (assuming it's the main camer

    return None  # Return None if no valid number is found

# Apply cleaning function
df['Back Camera'] = df['Back Camera'].apply(clean_back_camera)
```

```
# Convert the column to numeric type
df['Back Camera'] = pd.to_numeric(df['Back Camera'])
```

In [405…

```
df['Back Camera'].value_counts()
```

Out[405…

```
Back Camera
50.0     367
64.0     120
48.0     105
13.0      98
12.0      73
108.0     55
8.0       37
16.0      27
200.0     13
5.0       11
12.2       9
20.0       4
40.0       4
54.0       3
160.0      2
100.0      2
Name: count, dtype: int64
```

In [406…

```
df.head()
```

Out[406…

	Company Name	Model Name	Mobile Weight(g)	RAM(GB)	Front Camera	Back Camera	Processor	Battery Capacity(mAh)	Si
0	Apple	iPhone 16 128GB	174.0	6.0	12.0	48.0	A17 Bionic	3600.0	
1	Apple	iPhone 16 256GB	174.0	6.0	12.0	48.0	A17 Bionic	3600.0	
2	Apple	iPhone 16 512GB	174.0	6.0	12.0	48.0	A17 Bionic	3600.0	
3	Apple	iPhone 16 Plus 128GB	203.0	6.0	12.0	48.0	A17 Bionic	4200.0	
4	Apple	iPhone 16 Plus 256GB	203.0	6.0	12.0	48.0	A17 Bionic	4200.0	

In [407…

```
df.describe()
```

Out[407…

	Mobile Weight(g)	RAM(GB)	Front Camera	Back Camera	Battery Capacity(mAh)	Screen Size(inches)	La Pri
count	930.000000	930.000000	930.000000	930.000000	930.000000	930.000000	929
mean	228.267097	7.789247	18.163011	46.854624	5026.163441	7.083796	125436
std	105.432503	3.181315	11.986228	31.068100	1355.548264	1.533690	101593
min	135.000000	1.000000	2.000000	5.000000	2000.000000	5.000000	15999
25%	185.000000	6.000000	8.000000	16.000000	4402.500000	6.500000	54999
50%	194.000000	8.000000	16.000000	50.000000	5000.000000	6.670000	85000
75%	208.000000	8.000000	32.000000	50.000000	5091.250000	6.780000	179999
max	732.000000	16.000000	60.000000	200.000000	11200.000000	14.600000	604999

In [408…

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 930 entries, 0 to 929
Data columns (total 15 columns):
 #   Column                 Non-Null Count  Dtype  
---  ------                 --------------  -----  
 0   Company Name           930 non-null    object 
 1   Model Name             930 non-null    object 
 2   Mobile Weight(g)       930 non-null    float64
 3   RAM(GB)                930 non-null    float64
 4   Front Camera           930 non-null    float64
 5   Back Camera            930 non-null    float64
 6   Processor              930 non-null    object 
 7   Battery Capacity(mAh)  930 non-null    float64
 8   Screen Size(inches)    930 non-null    float64
 9   Launched Price(PKR)    929 non-null    float64
 10  Launched Price(INR)    930 non-null    float64
 11  Launched Price(CNY)    930 non-null    float64
 12  Launched Price(USD)    930 non-null    float64
 13  Launched Price(AED)    930 non-null    float64
 14  Launched Year          930 non-null    int64  
dtypes: float64(11), int64(1), object(3)
memory usage: 109.1+ KB
```

---

11. Univariate,Bivariate and Multivariate Analysis of colomns in the df dataset.

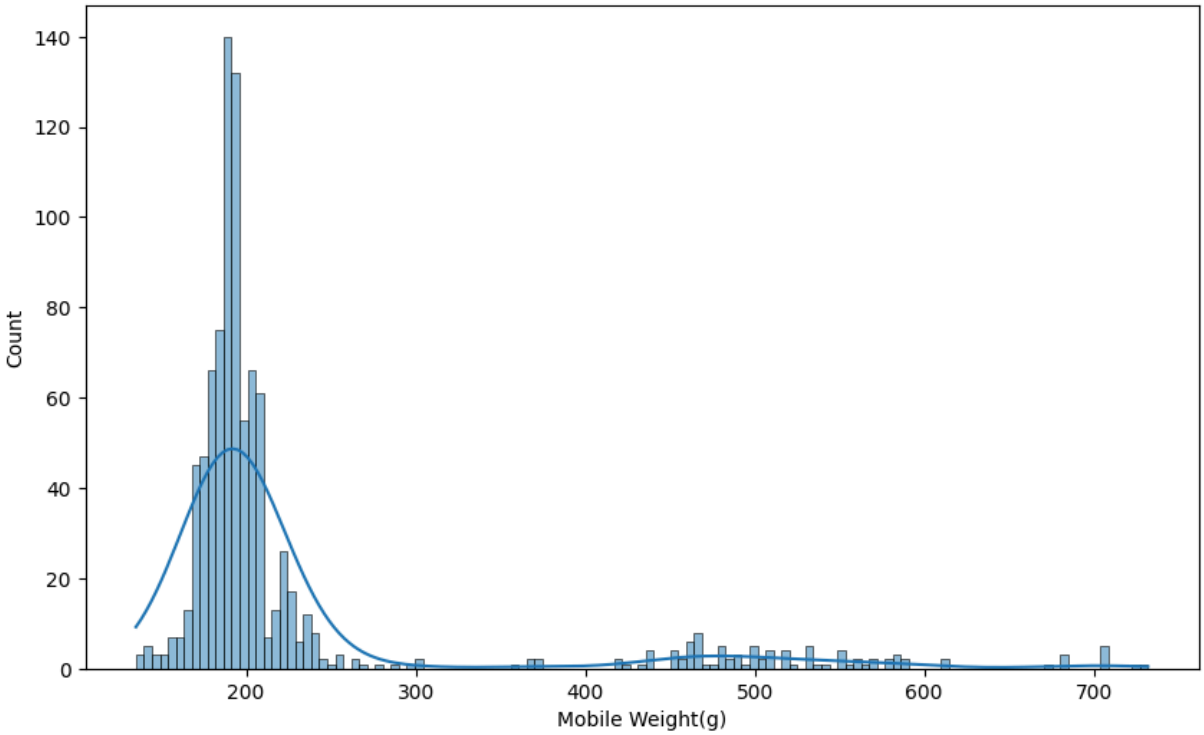
I. Univariate Analysis of:

(Mobile Weight(g),RAM(GB),Front Camera,Back Camera,Battery Capacity(mAh),Screen Size(inches), Launched Price(PKR),Launched Price(INR),Launched Price(CNY),Launched Price(USD),Launched Price(AED),Launched Year)

Numerical Colomns

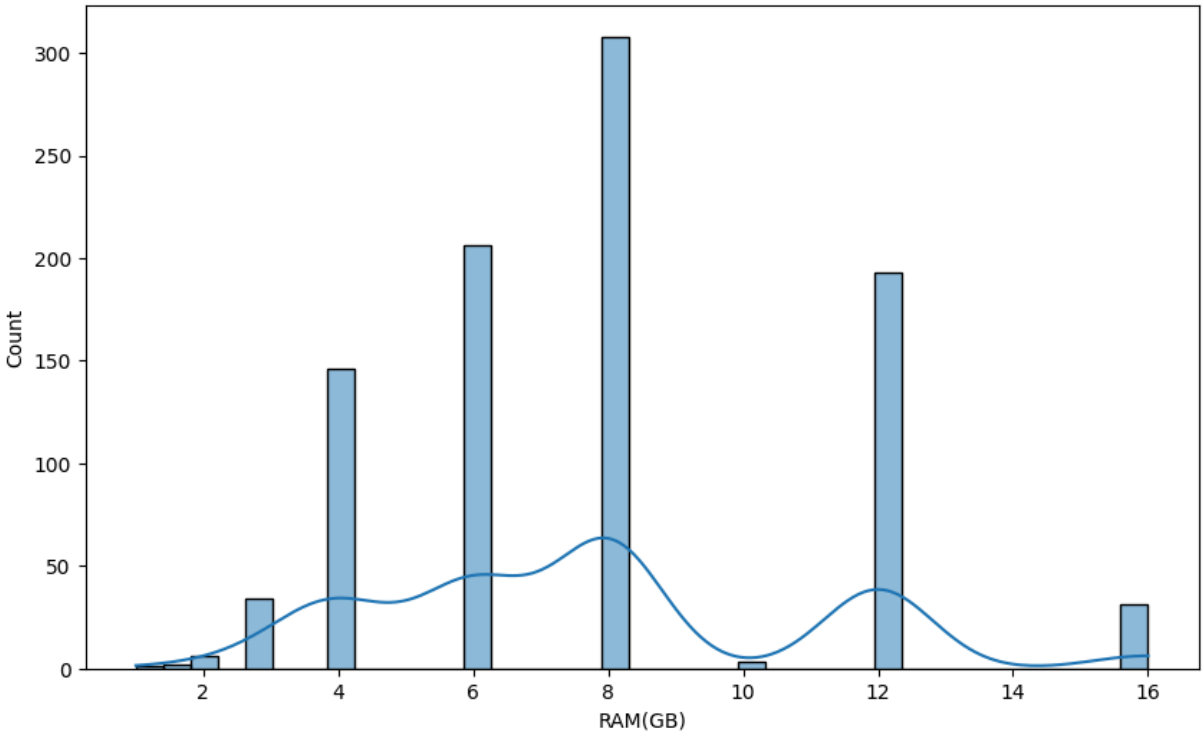
In [409…

```
# For Mobile Weight(g)
plt.figure(figsize=(10,6))
sns.histplot(df['Mobile Weight(g)'],kde=True)
plt.xlabel('Mobile Weight(g)')
plt.show()
```

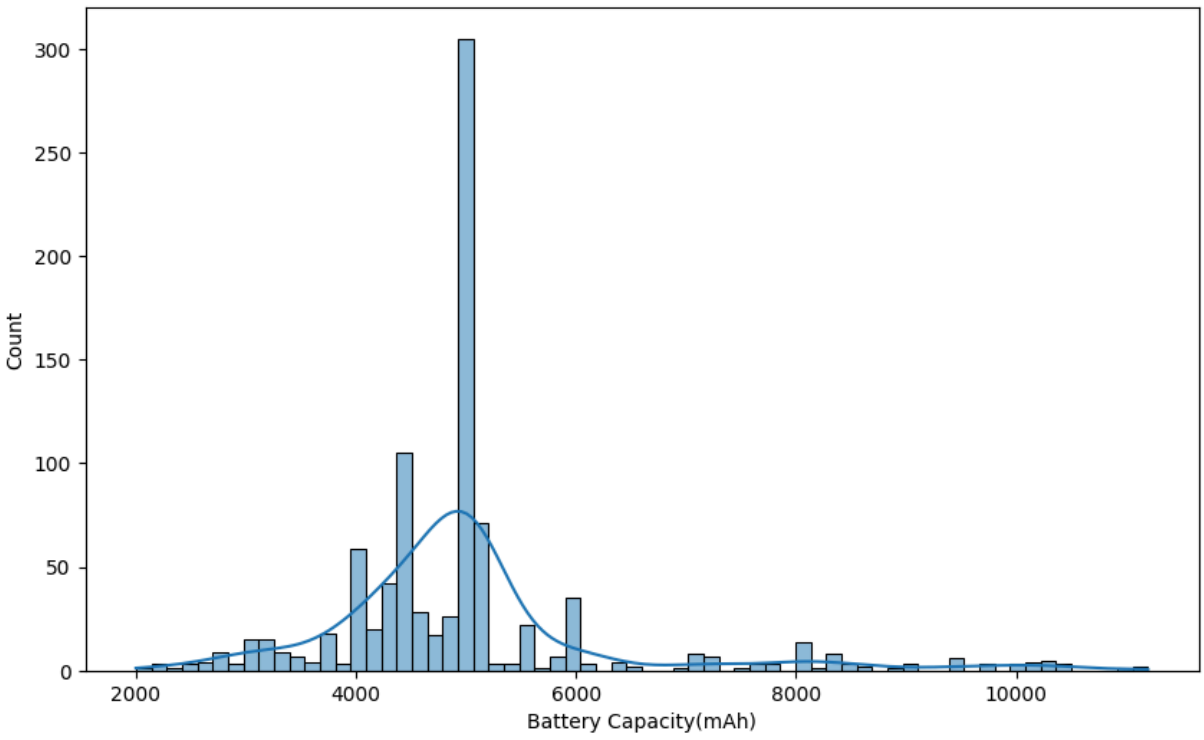


In [410…

```
# For RAM(GB)
plt.figure(figsize=(10,6))
sns.histplot(df['RAM(GB)'],kde=True)
plt.xlabel('RAM(GB)')
plt.show()
```

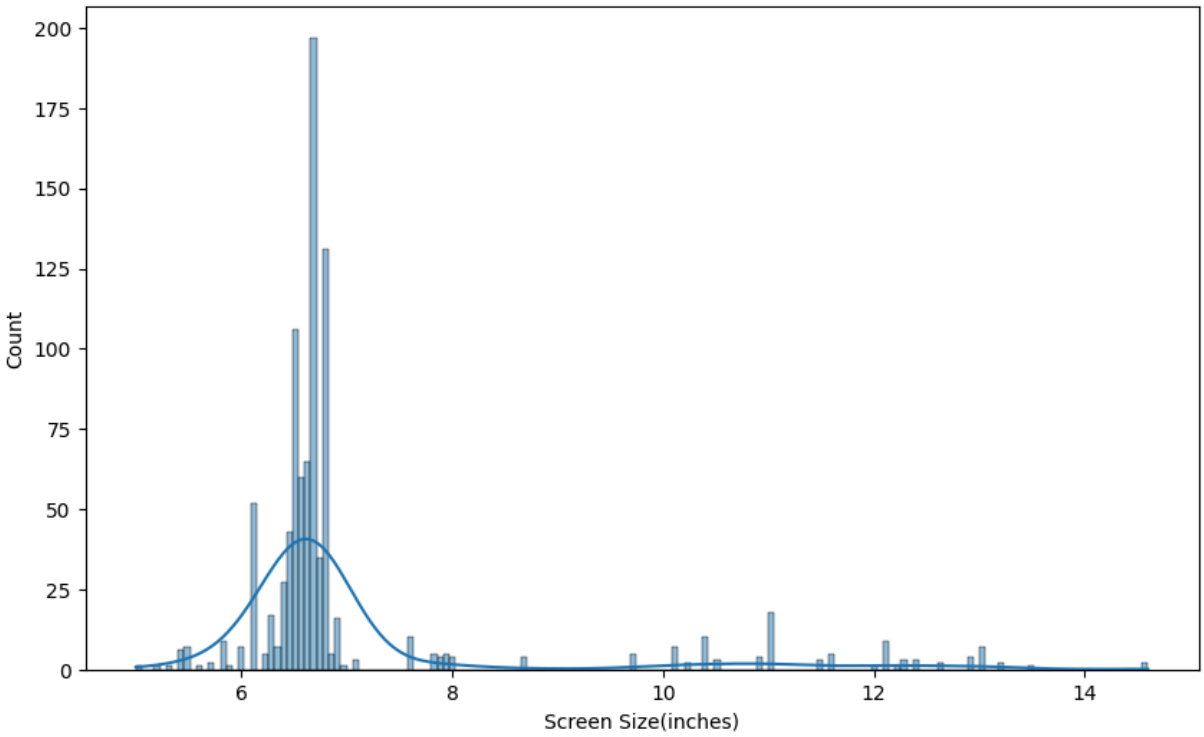


In [411…
```
# For Battery Capacity(mAh)
plt.figure(figsize=(10,6))
sns.histplot(df['Battery Capacity(mAh)'],kde=True)
plt.xlabel('Battery Capacity(mAh)')
plt.show()
```



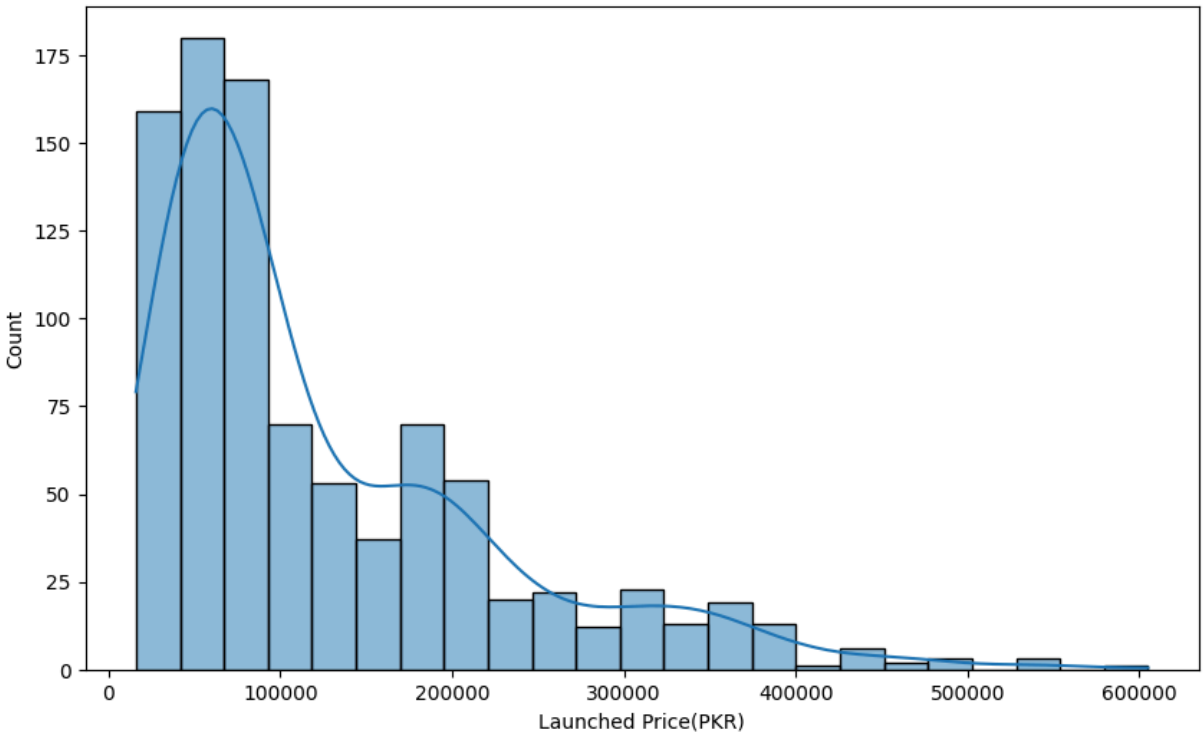
In [412…
```
# For Screen Size(inches)
plt.figure(figsize=(10,6))
sns.histplot(df['Screen Size(inches)'],kde=True)
```

```
plt.xlabel('Screen Size(inches)')
plt.show()
```



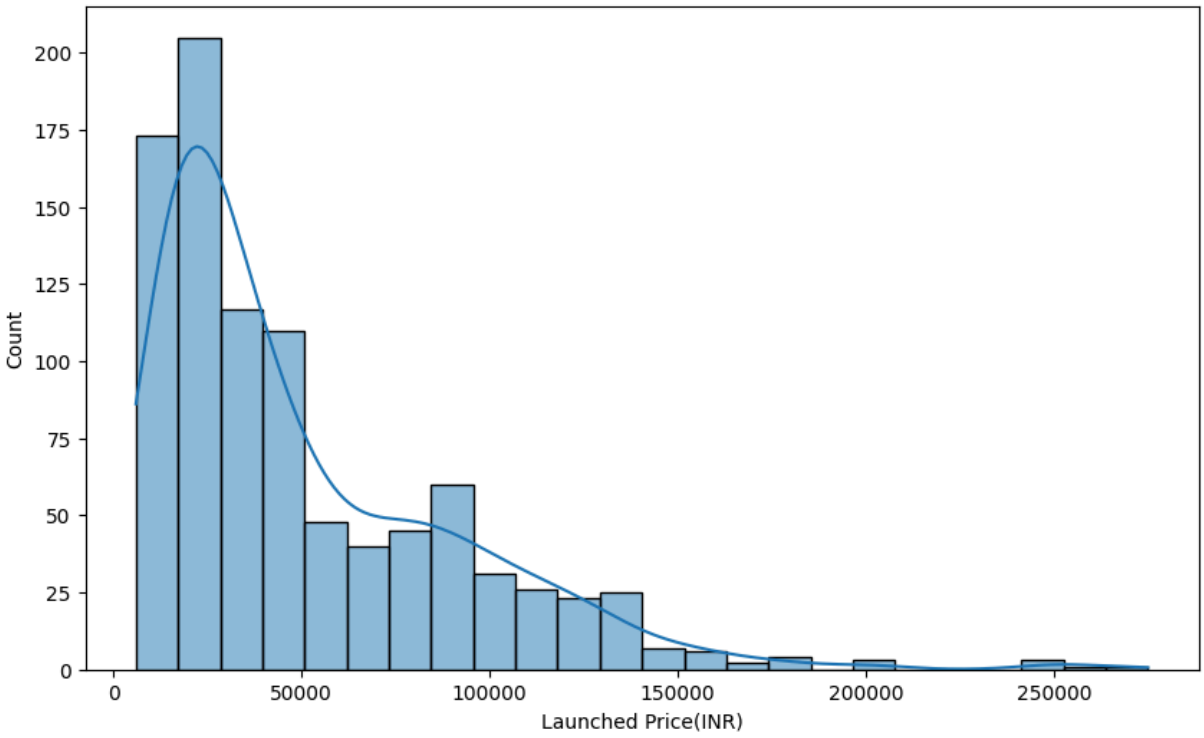
In [413…

```
# For Launched Price(PKR)
plt.figure(figsize=(10,6))
sns.histplot(df['Launched Price(PKR)'],kde=True)
plt.xlabel('Launched Price(PKR)')
plt.show()
```



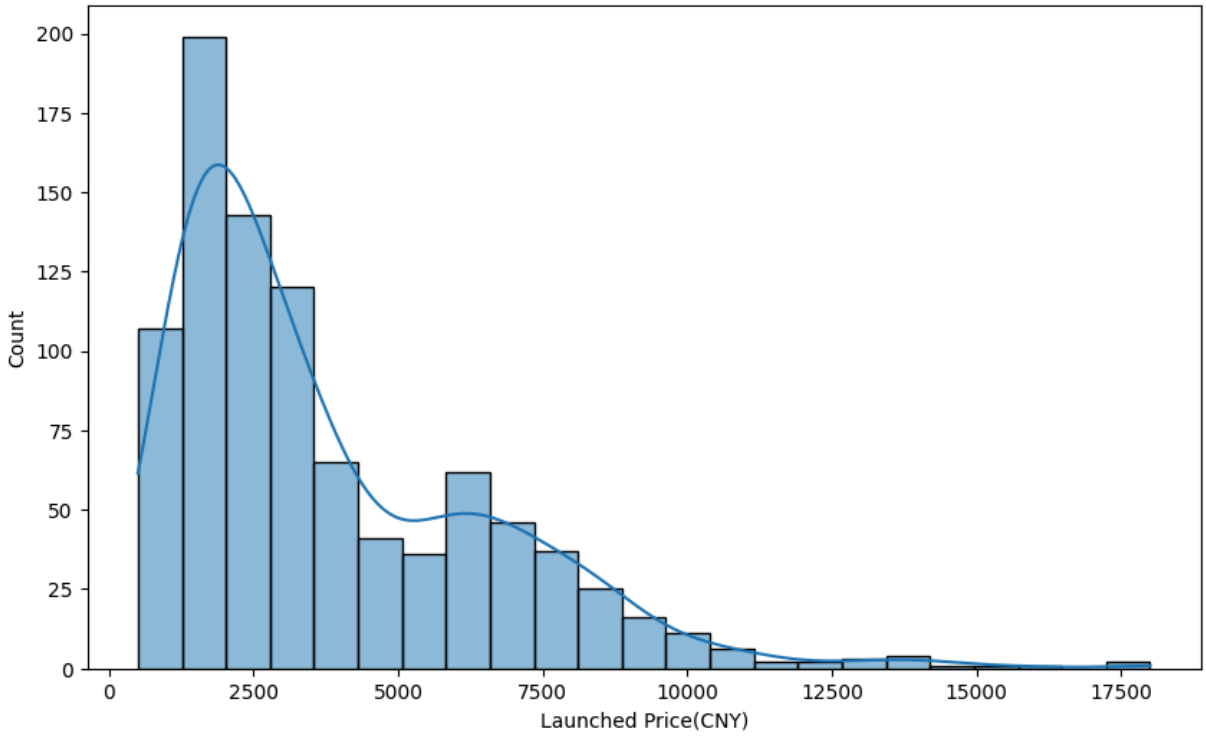
In [414…

```
# For Launched Price(INR)
plt.figure(figsize=(10,6))
sns.histplot(df['Launched Price(INR)'],kde=True)
plt.xlabel('Launched Price(INR)')
plt.show()
```



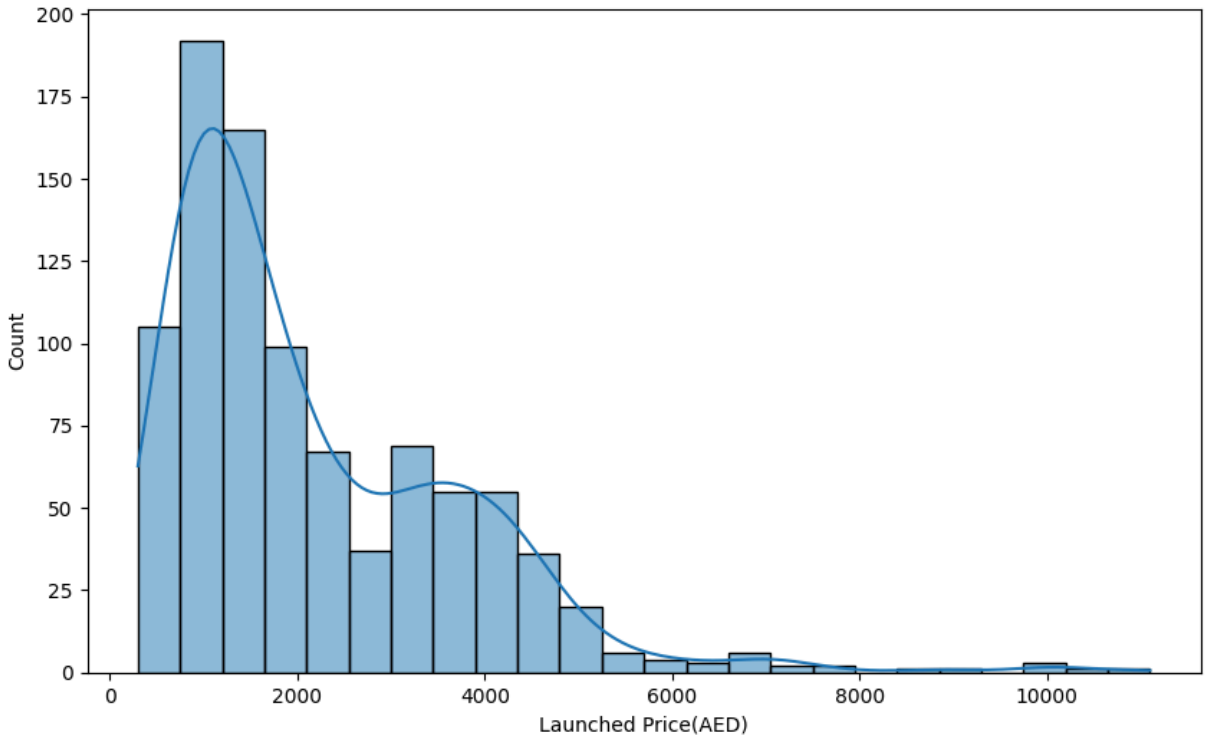
In [415…

```
# For Launched Price(CNY)
plt.figure(figsize=(10,6))
sns.histplot(df['Launched Price(CNY)'],kde=True)
plt.xlabel('Launched Price(CNY)')
plt.show()
```



In [416…

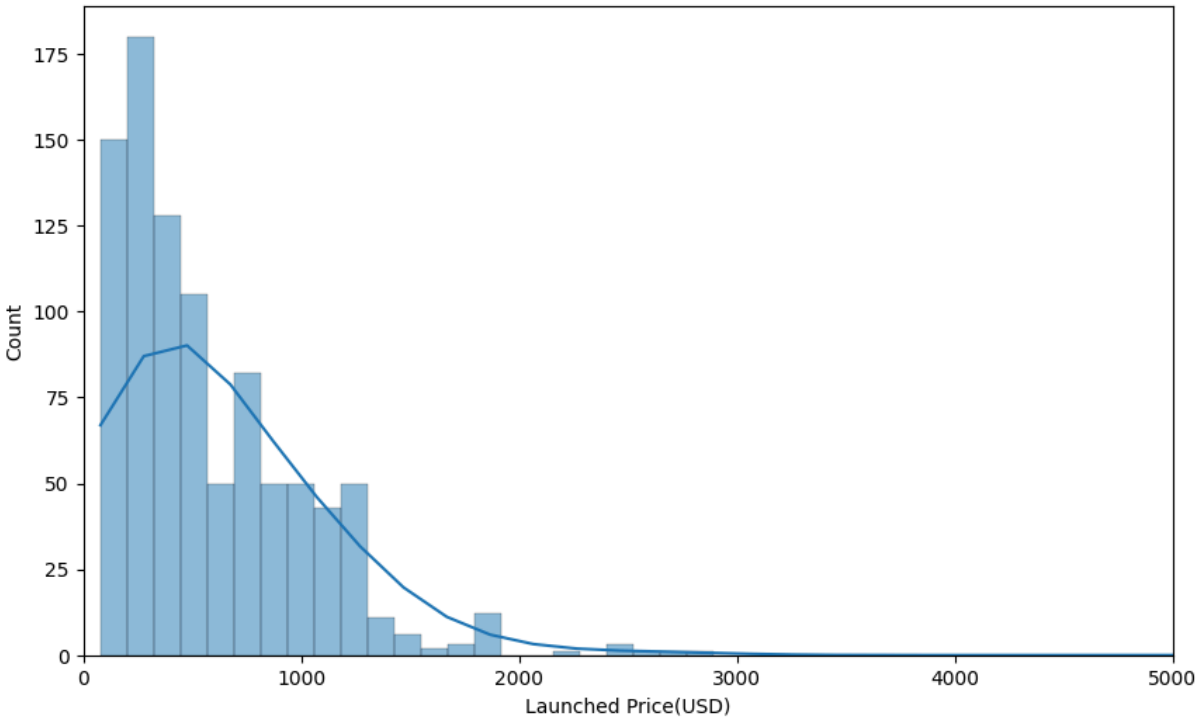
```
# For Launched Price(AED)
plt.figure(figsize=(10,6))
sns.histplot(df['Launched Price(AED)'],kde=True)
plt.xlabel('Launched Price(AED)')
plt.show()
```



In [417…

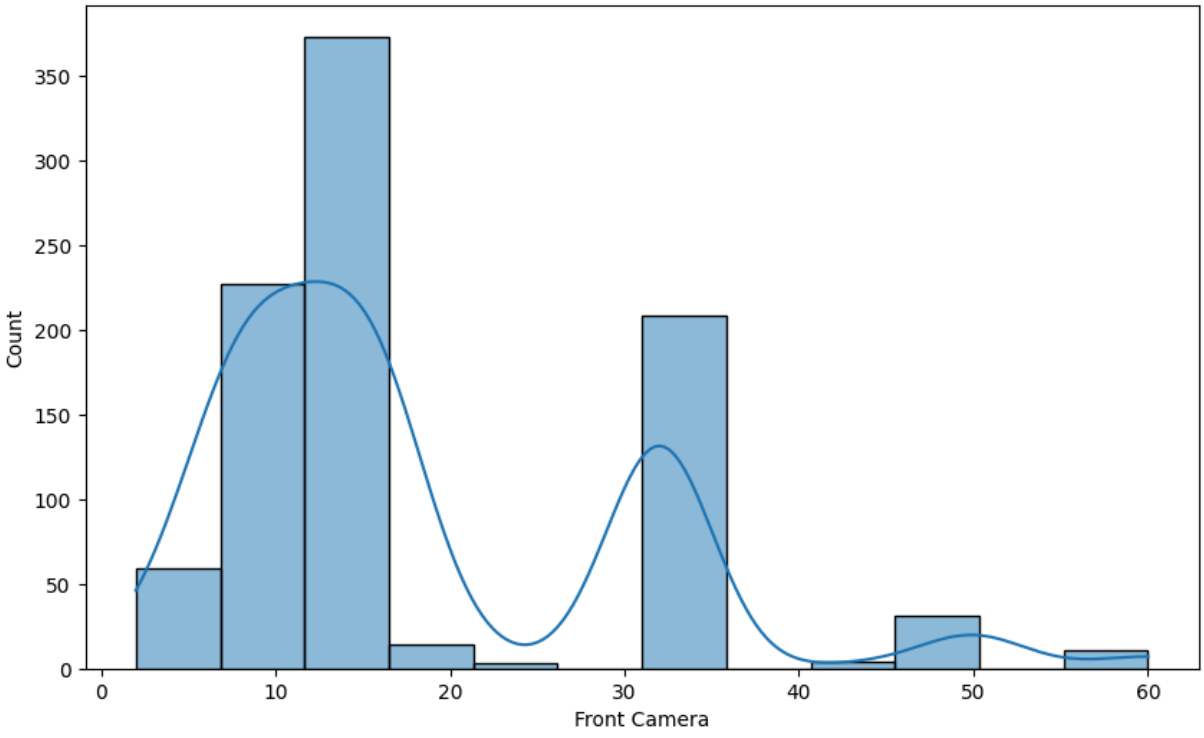
```
# For Launched Price(USD)
plt.figure(figsize=(10,6))
sns.histplot(df['Launched Price(USD)'], kde=True)
plt.xlabel('Launched Price(USD)')
```

```
plt.xlim(0, 5000)
plt.show()
```



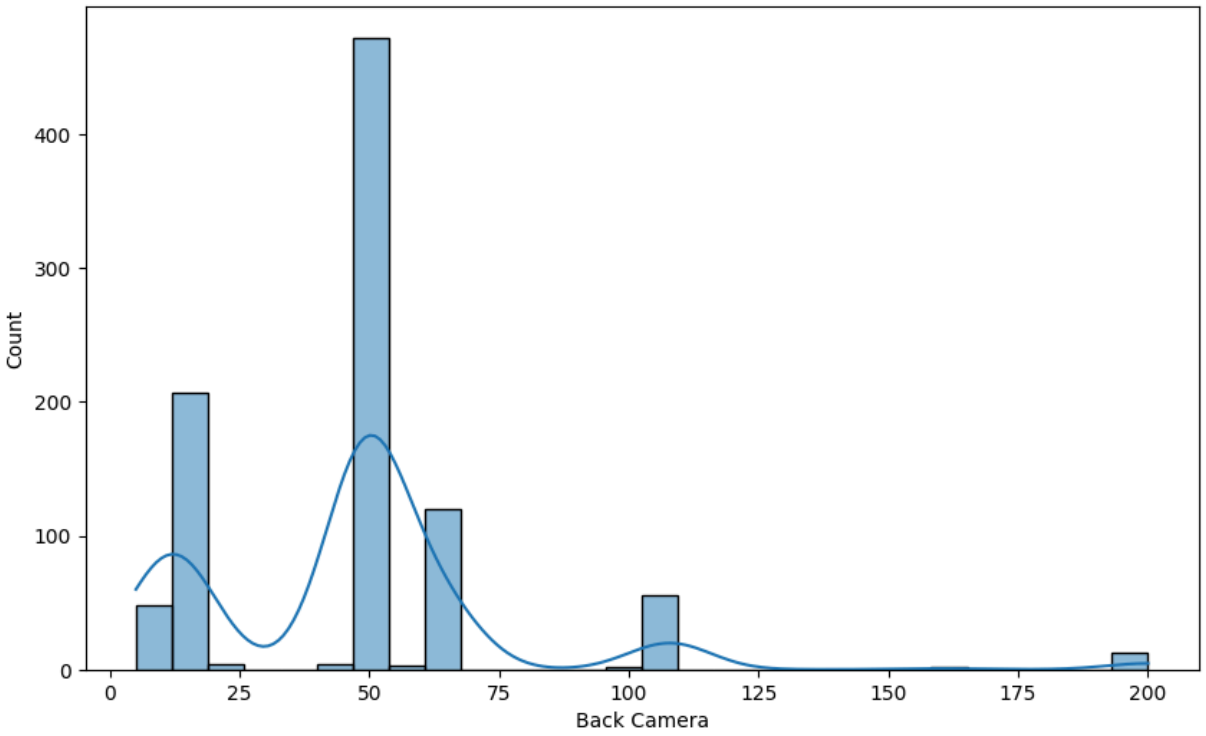
In [418…

```
# For Front Camera
plt.figure(figsize=(10,6))
sns.histplot(df['Front Camera'], kde=True)
plt.xlabel('Front Camera')
plt.show()
```



In [419…

```
# For Back Camera
plt.figure(figsize=(10,6))
sns.histplot(df['Back Camera'], kde=True)
plt.xlabel('Back Camera')
plt.show()
```



In [420…

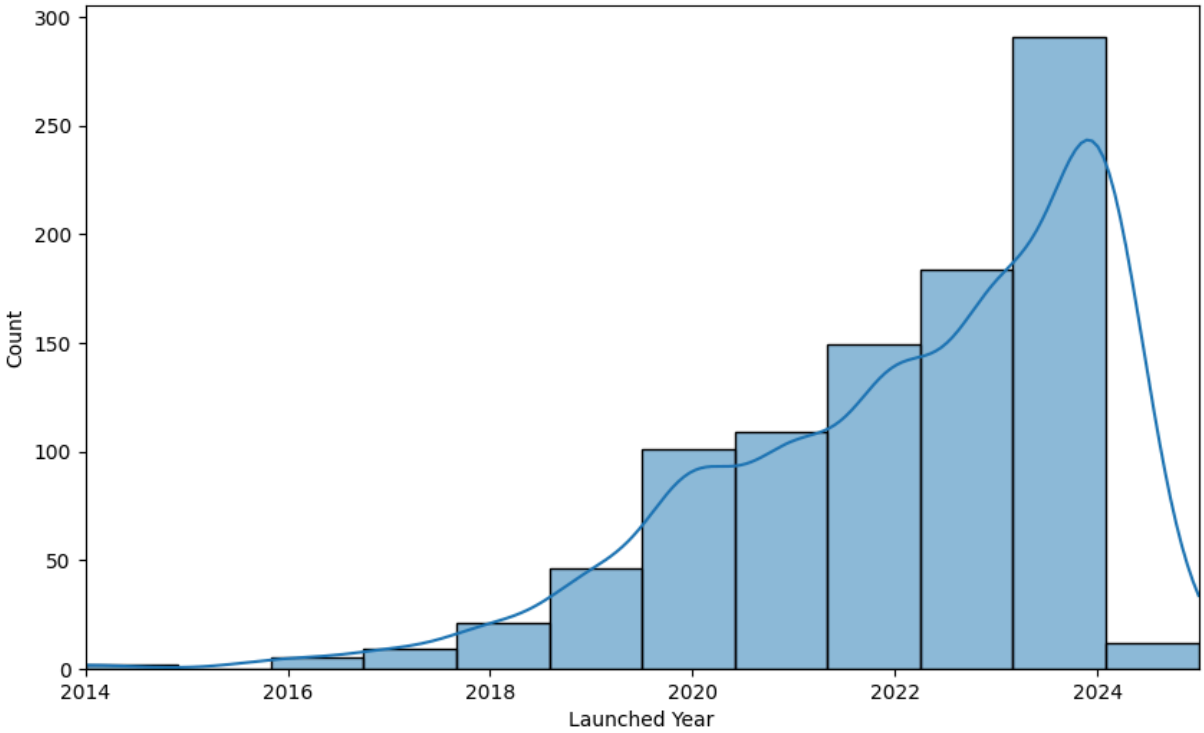
```
df['Launched Year'].unique()
```

Out[420…

```
array([2024, 2023, 2022, 2021, 2020, 2019, 2017, 2018, 2924, 2016, 2014,
       2025])
```

In [421…

```
# Filter the dataset for the years 2014 to 2025
filtered_df = df[(df['Launched Year'] >= 2014) & (df['Launched Year'] <= 2025)]

# Plot the histogram
plt.figure(figsize=(10,6))
sns.histplot(filtered_df['Launched Year'], kde=True, bins=12)
plt.xlabel('Launched Year')
plt.xlim(2014, 2025)
plt.show()
```



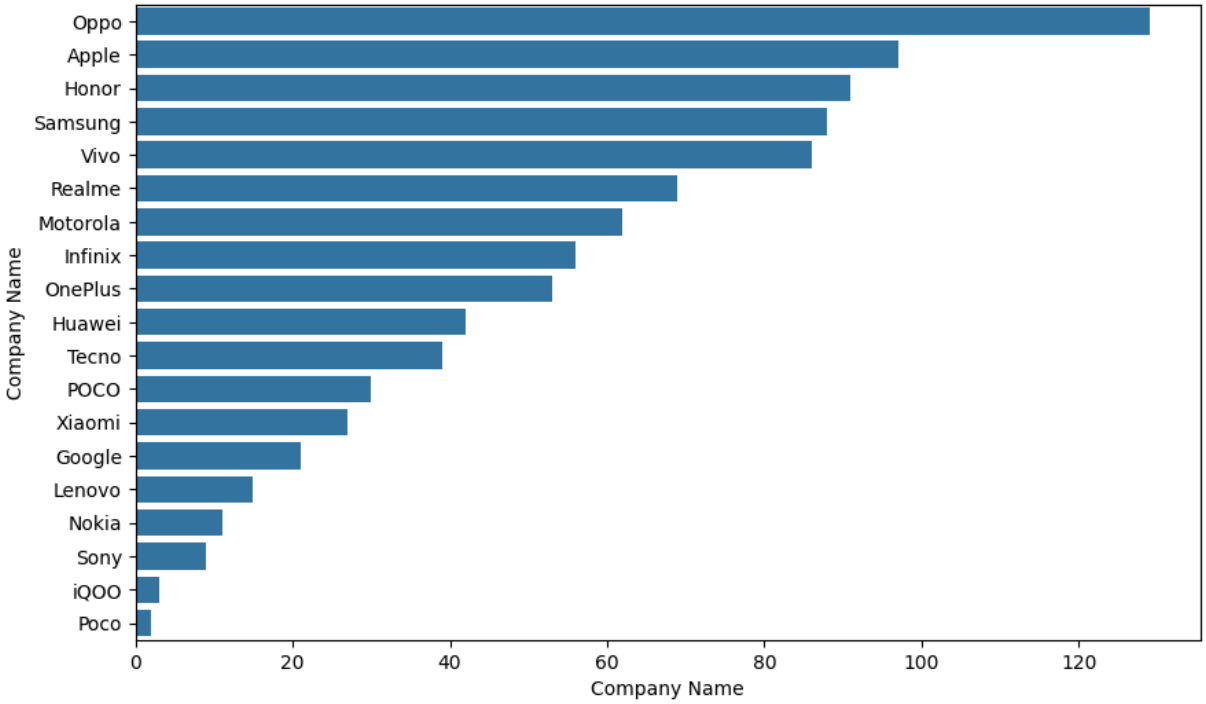
Univariate Analysis of:

(Company Name,Model Name,Processor)

Categorical Colomns

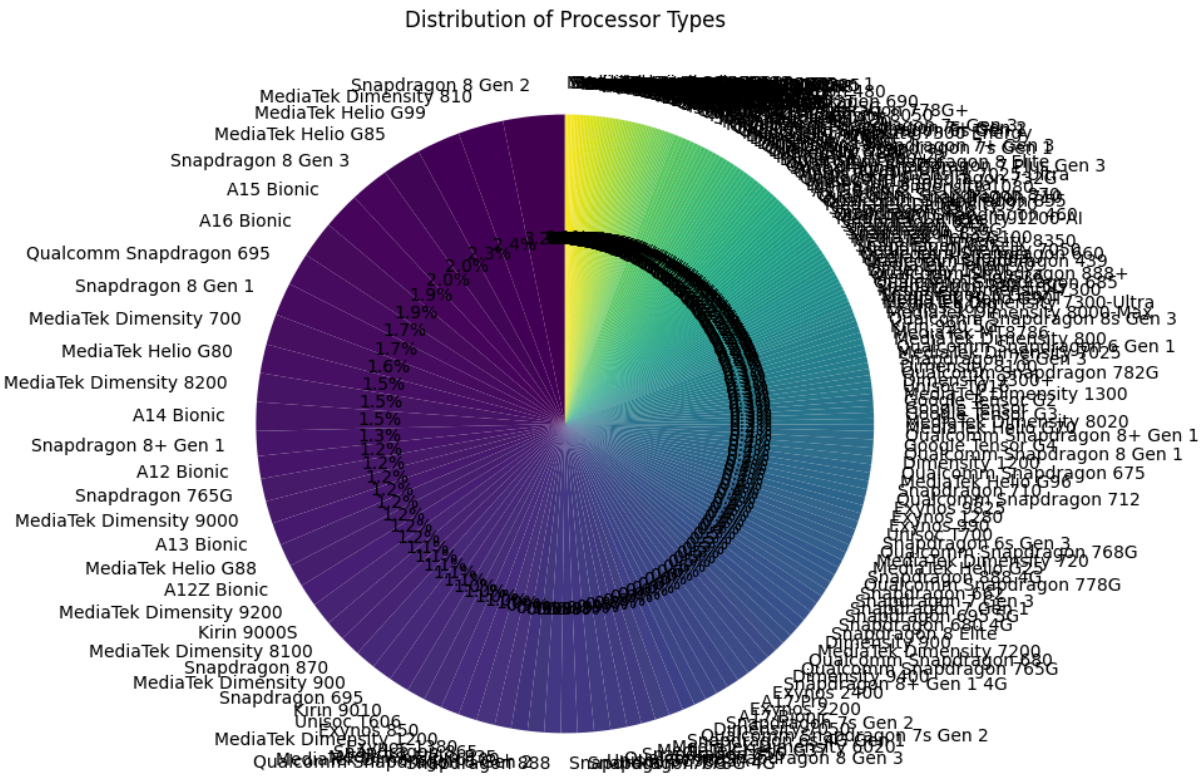
In [422…

```
# For Company Name, in assending order
plt.figure(figsize=(10,6))
sns.countplot(y='Company Name', data=df, order=df['Company Name'].value_counts().in
plt.xlabel('Company Name')
plt.show()
```



In [423…

```
# Count the occurrences of each processor type
processor_counts = df['Processor'].value_counts()

# Plot the pie chart
plt.figure(figsize=(10, 8))
processor_counts.plot.pie(autopct='%1.1f%%', startangle=90, cmap='viridis')
plt.ylabel('')  # Hide the y-label
plt.title('Distribution of Processor Types')
plt.show()
```



II. Bivariate Analysis of Colomns in the df dataset

I. Numerical Variables.

(Mobile Weight(g),RAM(GB),Front Camera,Back Camera,Battery Capacity(mAh),Screen Size(inches), Launched Price(PKR),Launched Price(INR),Launched Price(CNY),Launched Price(USD),Launched Price(AED),Launched Year)

In [424…

```
# Find the correlation of the numeric columns in the df dataset
numeric_df = df.select_dtypes(include=['number'])
correlation_matrix = numeric_df.corr()
correlation_matrix
```

Out[424...

	Mobile Weight(g)	RAM(GB)	Front Camera	Back Camera	Battery Capacity(mAh)	Screen Size(inches)	L Pr
Mobile Weight(g)	1.000000	-0.007748	-0.283264	-0.309245	0.847731	0.975893	
RAM(GB)	-0.007748	1.000000	0.459273	0.443776	0.139072	0.046600	
Front Camera	-0.283264	0.459273	1.000000	0.451230	-0.159587	-0.224311	
Back Camera	-0.309245	0.443776	0.451230	1.000000	-0.106936	-0.263684	
Battery Capacity(mAh)	0.847731	0.139072	-0.159587	-0.106936	1.000000	0.879453	
Screen Size(inches)	0.975893	0.046600	-0.224311	-0.263684	0.879453	1.000000	
Launched Price(PKR)	0.101551	0.414087	-0.016261	0.084789	-0.040874	0.064332	
Launched Price(INR)	0.141899	0.423889	0.023723	0.038314	-0.007829	0.108592	
Launched Price(CNY)	0.137530	0.429856	0.026119	0.041094	-0.023587	0.107392	
Launched Price(USD)	0.104919	0.109066	-0.003924	-0.013129	0.063929	0.095838	
Launched Price(AED)	0.099842	0.479745	0.065004	0.093280	-0.037953	0.080205	
Launched Year	-0.005837	0.026089	-0.006376	0.023303	0.016664	-0.001566	

In [425...

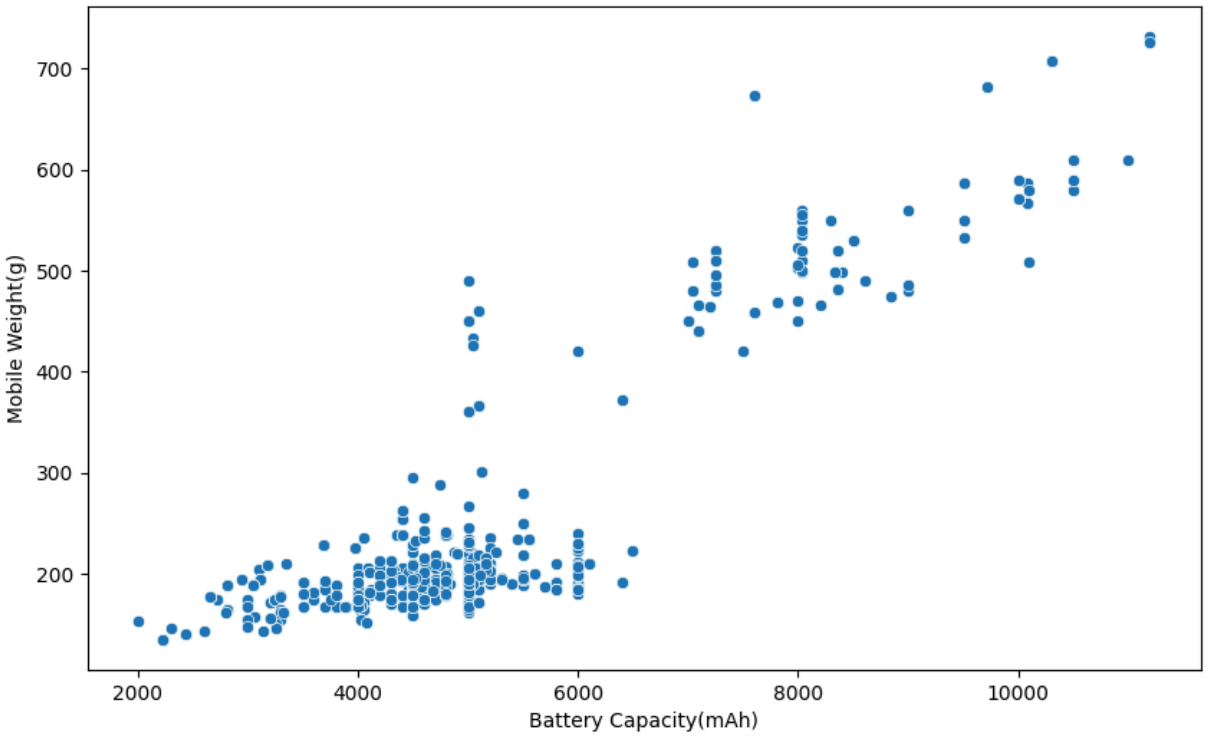
```
# Set a threshold for high correlation
high_corr_threshold = 0.5

# Find the correlation of the numeric columns in the df dataset
numeric_df = df.select_dtypes(include=['number'])
correlation_matrix = numeric_df.corr()

# Find pairs of highly correlated columns
high_corr_pairs = correlation_matrix[(correlation_matrix.abs() > high_corr_threshol

# Drop NaN values
high_corr_pairs = high_corr_pairs.dropna(how='all').dropna(axis=1, how='all')

high_corr_pairs
```

Out[425...

	Mobile Weight(g)	Battery Capacity(mAh)	Screen Size(inches)	Launched Price(PKR)	Launched Price(INR)	Launched Price(CNY)
Mobile Weight(g)	NaN	0.847731	0.975893	NaN	NaN	NaN
Battery Capacity(mAh)	0.847731	NaN	0.879453	NaN	NaN	NaN
Screen Size(inches)	0.975893	0.879453	NaN	NaN	NaN	NaN
Launched Price(PKR)	NaN	NaN	NaN	NaN	0.904178	0.902995
Launched Price(INR)	NaN	NaN	NaN	0.904178	NaN	0.966548
Launched Price(CNY)	NaN	NaN	NaN	0.902995	0.966548	NaN
Launched Price(AED)	NaN	NaN	NaN	0.898456	0.969457	0.969246

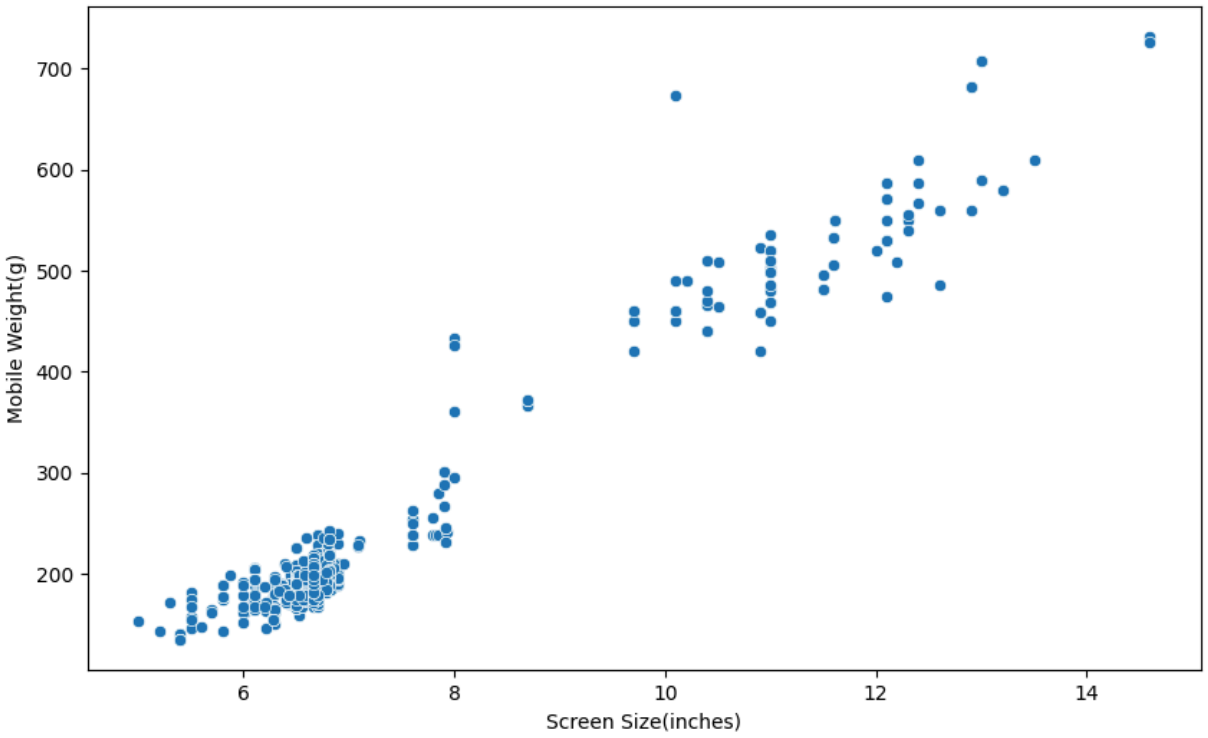
In [426...

```
# For Battery Capacity(mAh) and Mobile Weight(g)
plt.figure(figsize=(10,6))
sns.scatterplot(x='Battery Capacity(mAh)', y='Mobile Weight(g)', data=df)
plt.xlabel('Battery Capacity(mAh)')
plt.ylabel('Mobile Weight(g)')
plt.show()
```



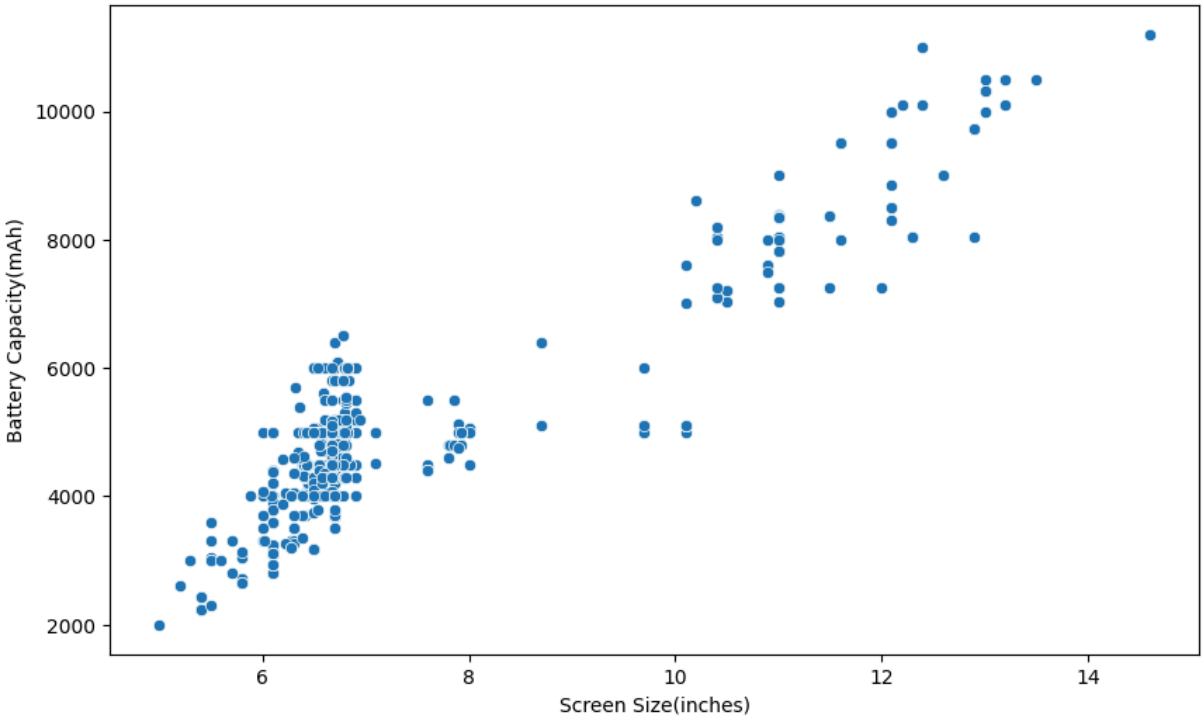
In [427…

```
# For Screen Size(inches) and Mobile Weight(g)
plt.figure(figsize=(10,6))
sns.scatterplot(x='Screen Size(inches)', y='Mobile Weight(g)', data=df)
plt.xlabel('Screen Size(inches)')
plt.ylabel('Mobile Weight(g)')
plt.show()
```



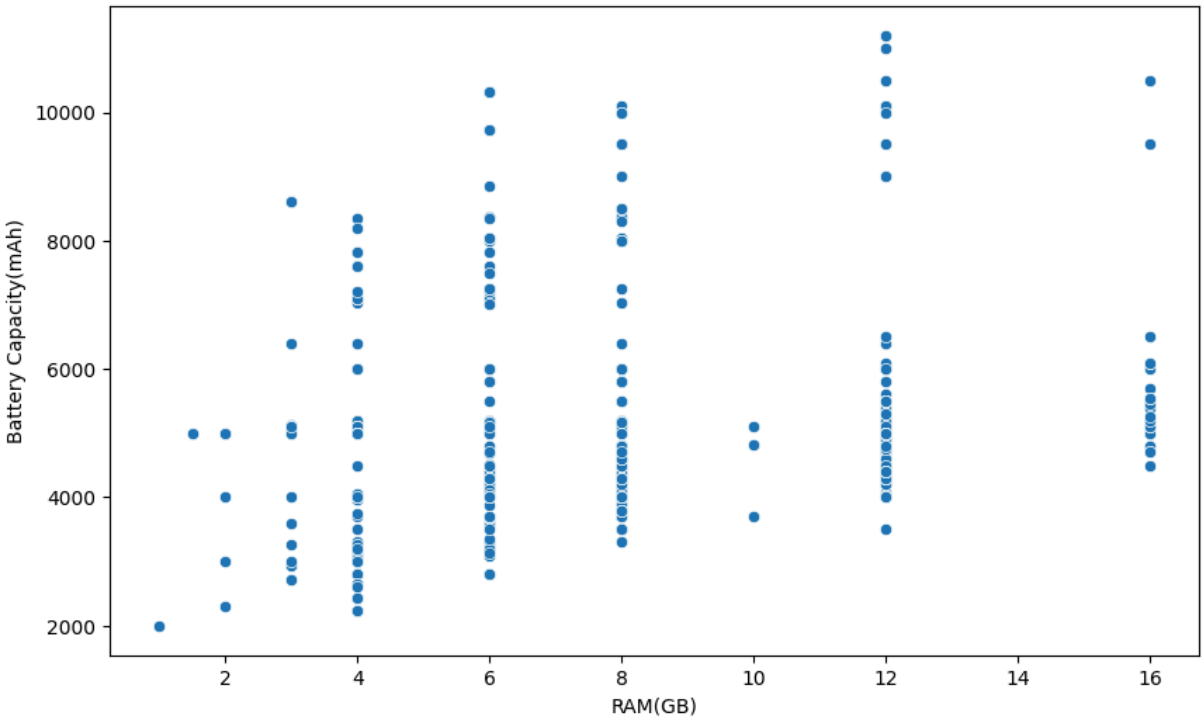
In [428…

```
# For Screen Size(inches) and Battery Capacity(mAh)
plt.figure(figsize=(10,6))
sns.scatterplot(x='Screen Size(inches)', y='Battery Capacity(mAh)', data=df)
plt.xlabel('Screen Size(inches)')
plt.ylabel('Battery Capacity(mAh)')
plt.show()
```



In [429…

```
# For RAM(GB) and Battery Capacity(mAh)
plt.figure(figsize=(10,6))
sns.scatterplot(x='RAM(GB)', y='Battery Capacity(mAh)', data=df)
plt.xlabel('RAM(GB)')
plt.ylabel('Battery Capacity(mAh)')
plt.show()
```

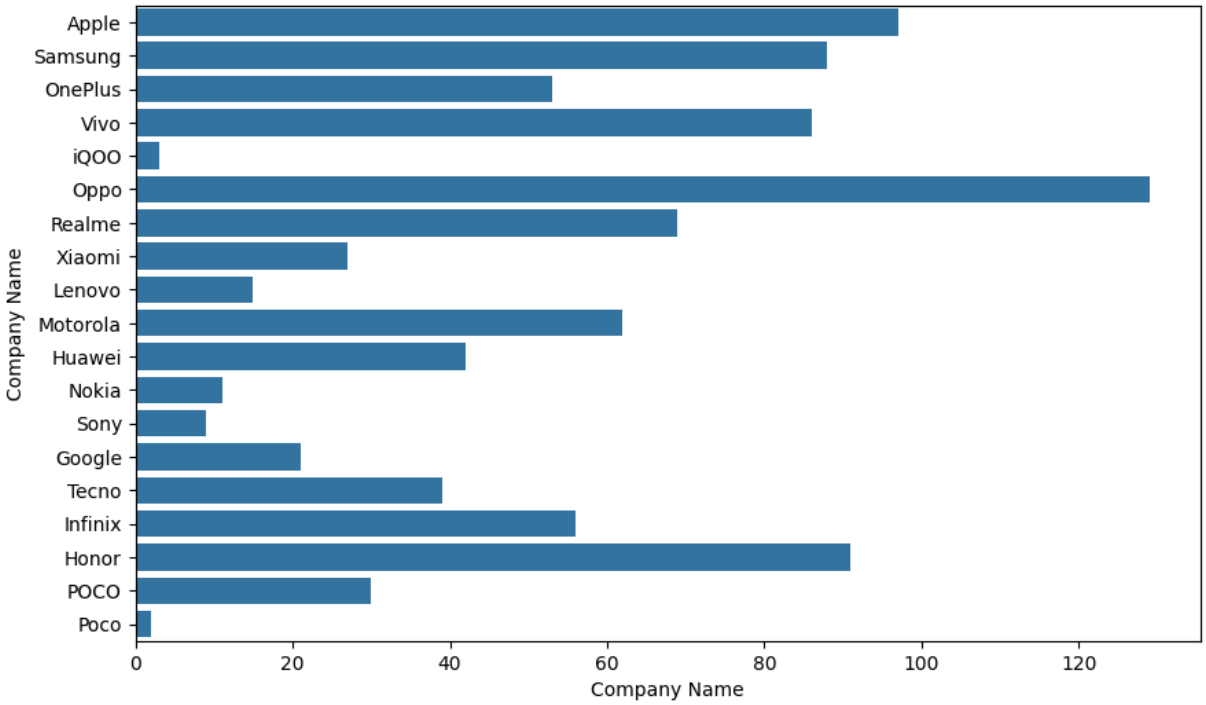


II. Categorical Variables

(**Company Name, Model Name, Processor**)

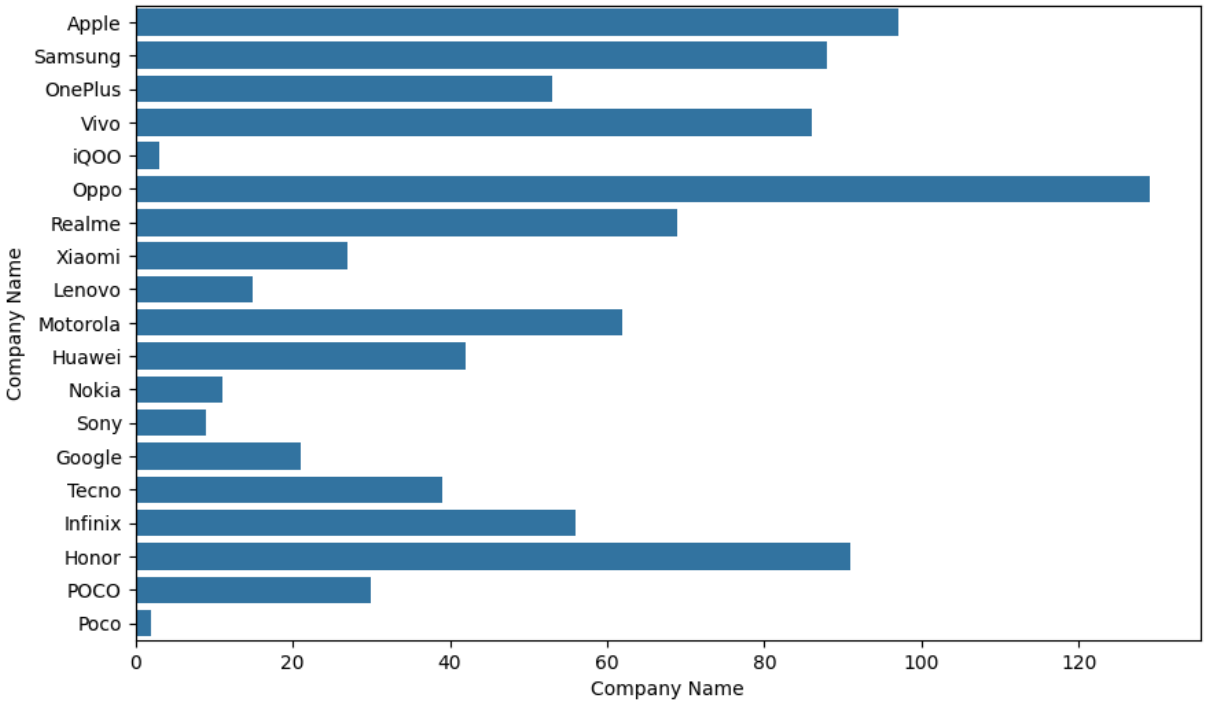
In [430…

```
# For Company Name and Model Name
plt.figure(figsize=(10,6))
sns.countplot(y='Company Name', data=df)
plt.xlabel('Company Name')
plt.show()
```



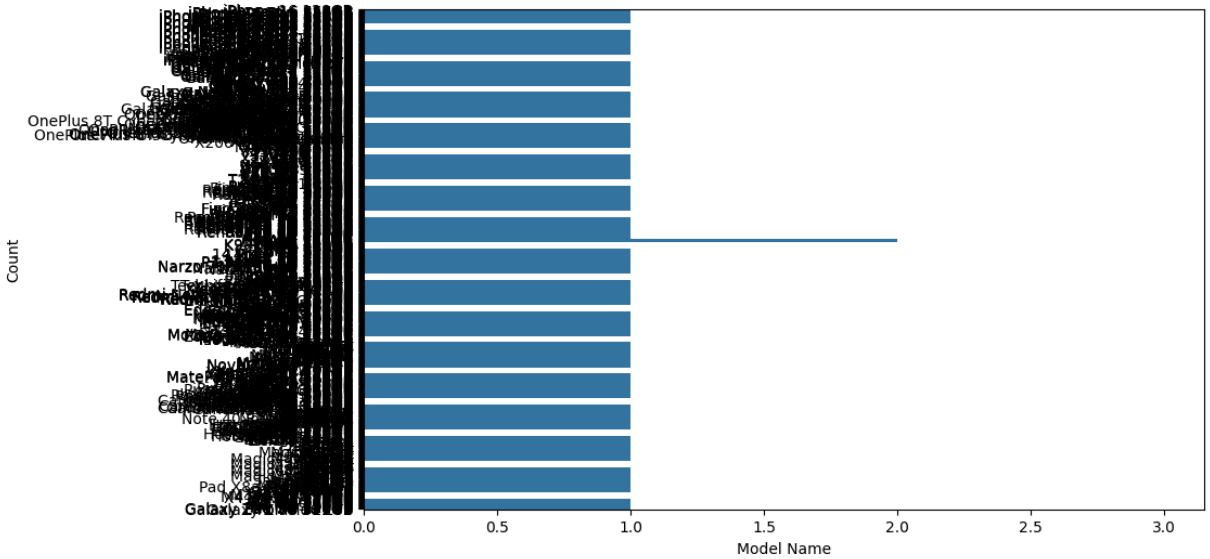
In [431…

```
# Check for the Company Name and Processor
plt.figure(figsize=(10,6))
sns.countplot(y='Company Name', data=df)
plt.xlabel('Company Name')
plt.show()
```



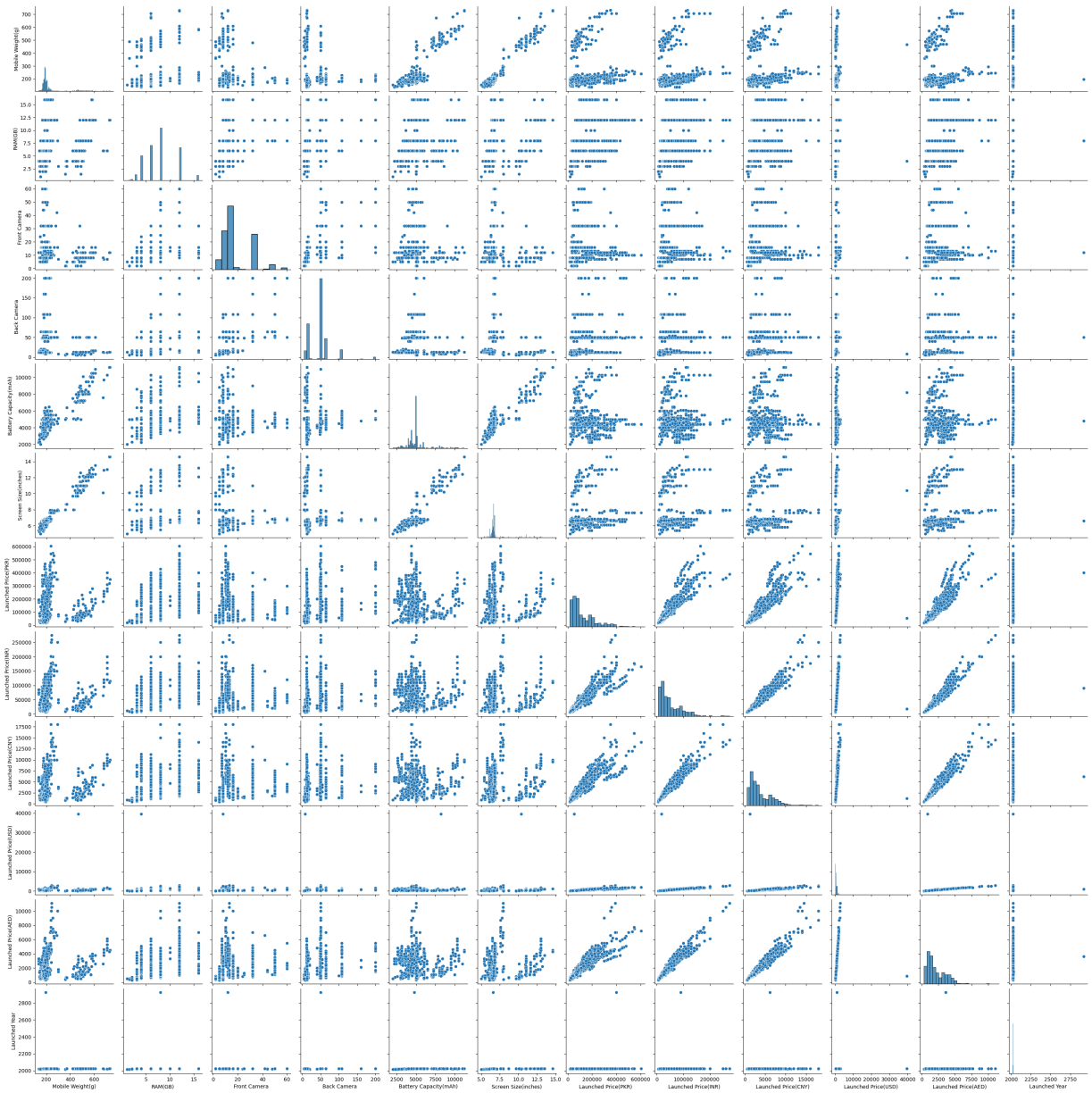
In [432…

```
# Check for the Model Name and Processor
plt.figure(figsize=(10,6))
sns.countplot(y='Model Name', data=df)
plt.xlabel('Model Name')
plt.ylabel('Count')
plt.show()
```



In [433…

```
# Create a pairplot of the numeric columns in the df dataset
sns.pairplot(df)
plt.show()
```



---

12. Detection of Outliers in the Numeric colomns in the df dataset.

In [434…

```
df.columns      # Check the column names
```

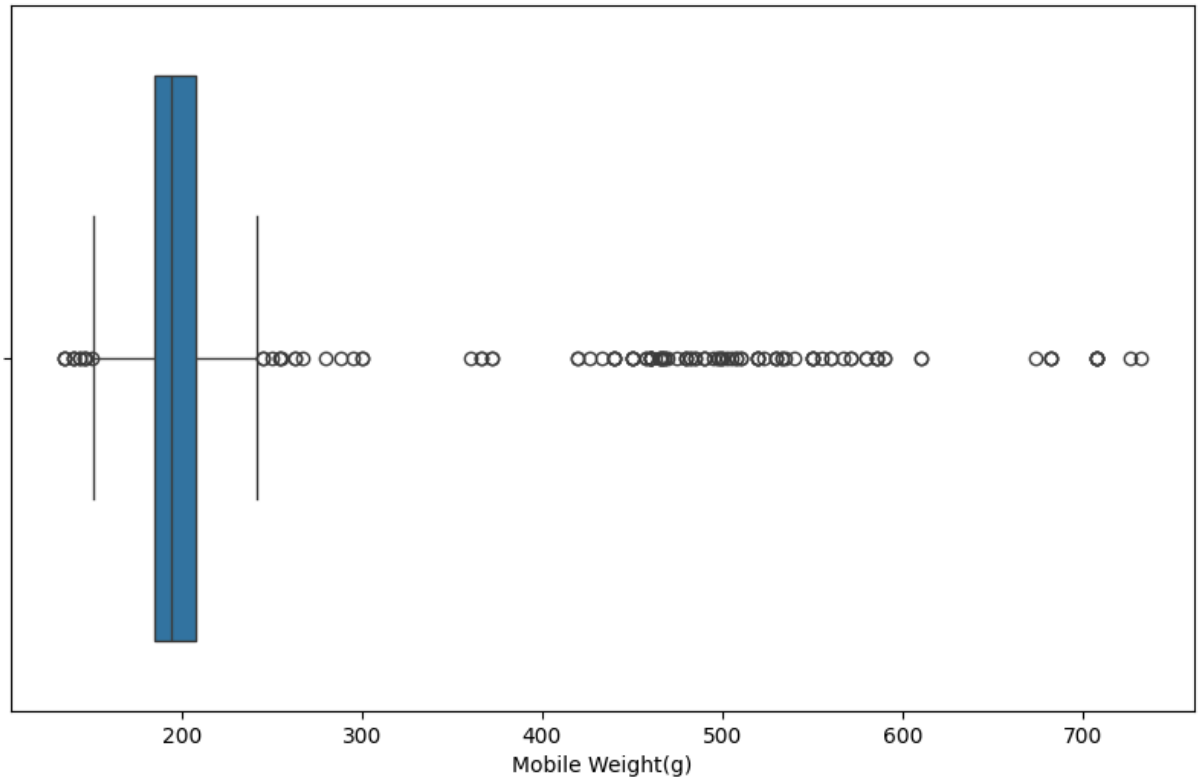
Out[434…

```
Index(['Company Name', 'Model Name', 'Mobile Weight(g)', 'RAM(GB)',
       'Front Camera', 'Back Camera', 'Processor', 'Battery Capacity(mAh)',
       'Screen Size(inches)', 'Launched Price(PKR)', 'Launched Price(INR)',
       'Launched Price(CNY)', 'Launched Price(USD)', 'Launched Price(AED)',
       'Launched Year'],
      dtype='object')
```

In [435…

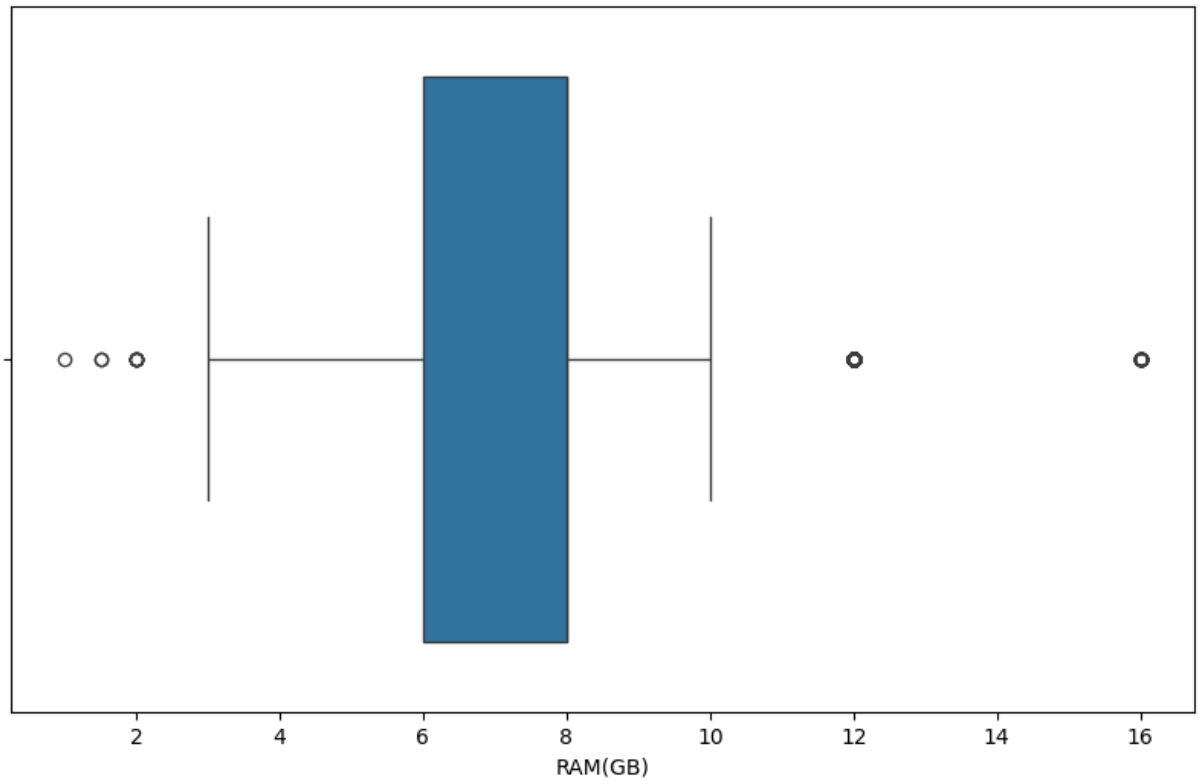
```
# Now check is there any outlier in the numeric colomns in the df dataset.
plt.figure(figsize=(10,6))
sns.boxplot(x='Mobile Weight(g)',data=df)
```

```
plt.xlabel('Mobile Weight(g)')
plt.show()
```



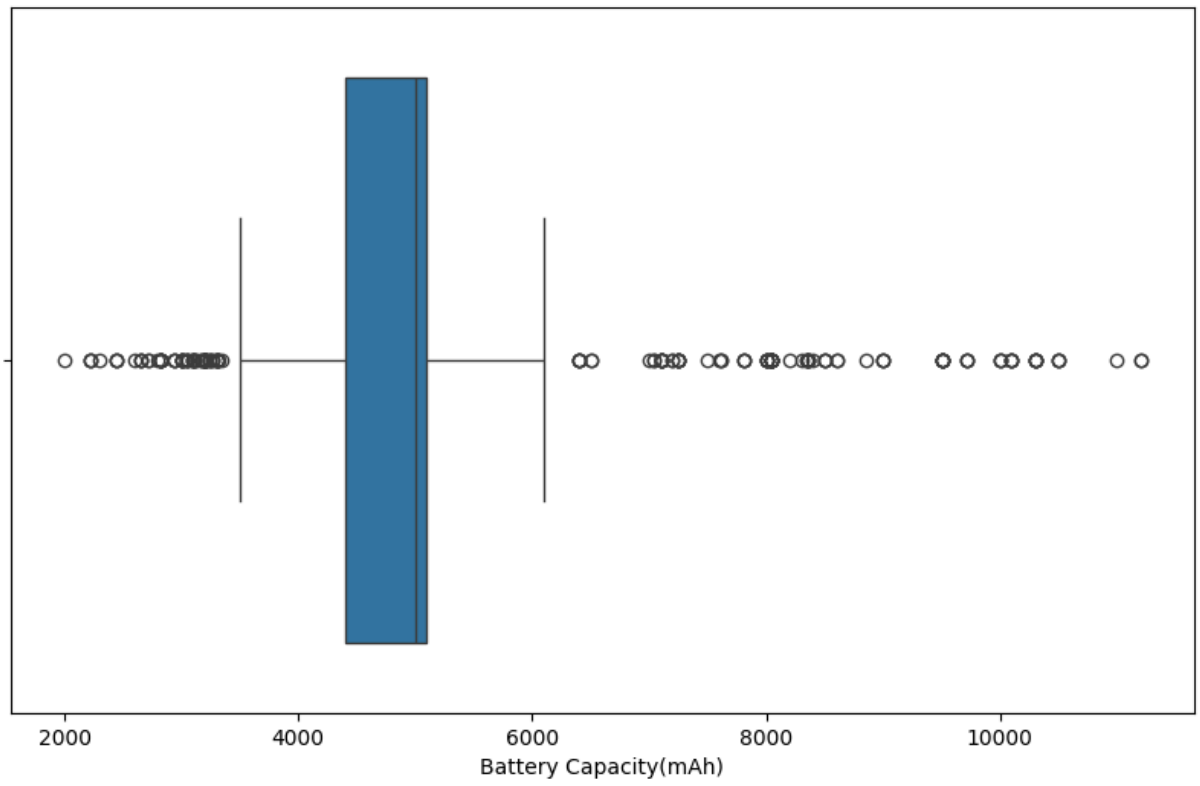
In [436…

```
# Check for the RAM(GB)
plt.figure(figsize=(10,6))
sns.boxplot(x='RAM(GB)',data=df)
plt.xlabel('RAM(GB)')
plt.show()
```



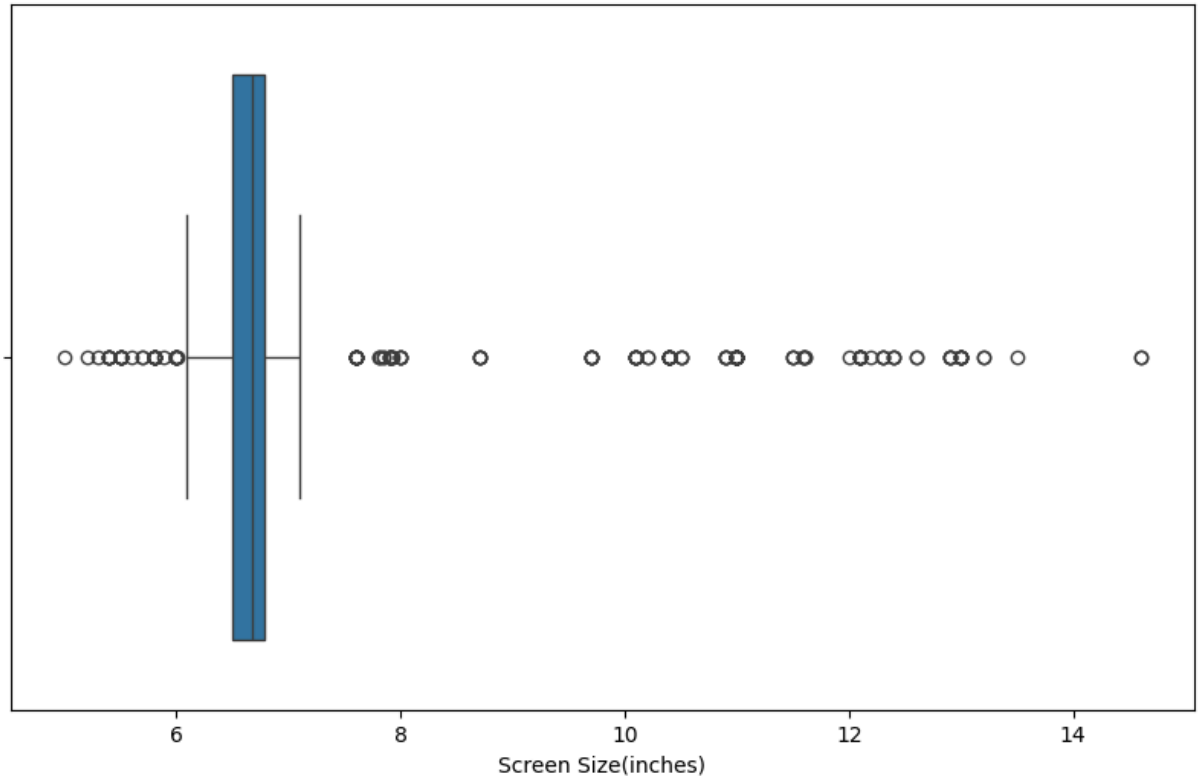
In [437…

```
# Check for the Battery Capacity(mAh)
plt.figure(figsize=(10,6))
sns.boxplot(x='Battery Capacity(mAh)',data=df)
plt.xlabel('Battery Capacity(mAh)')
plt.show()
```



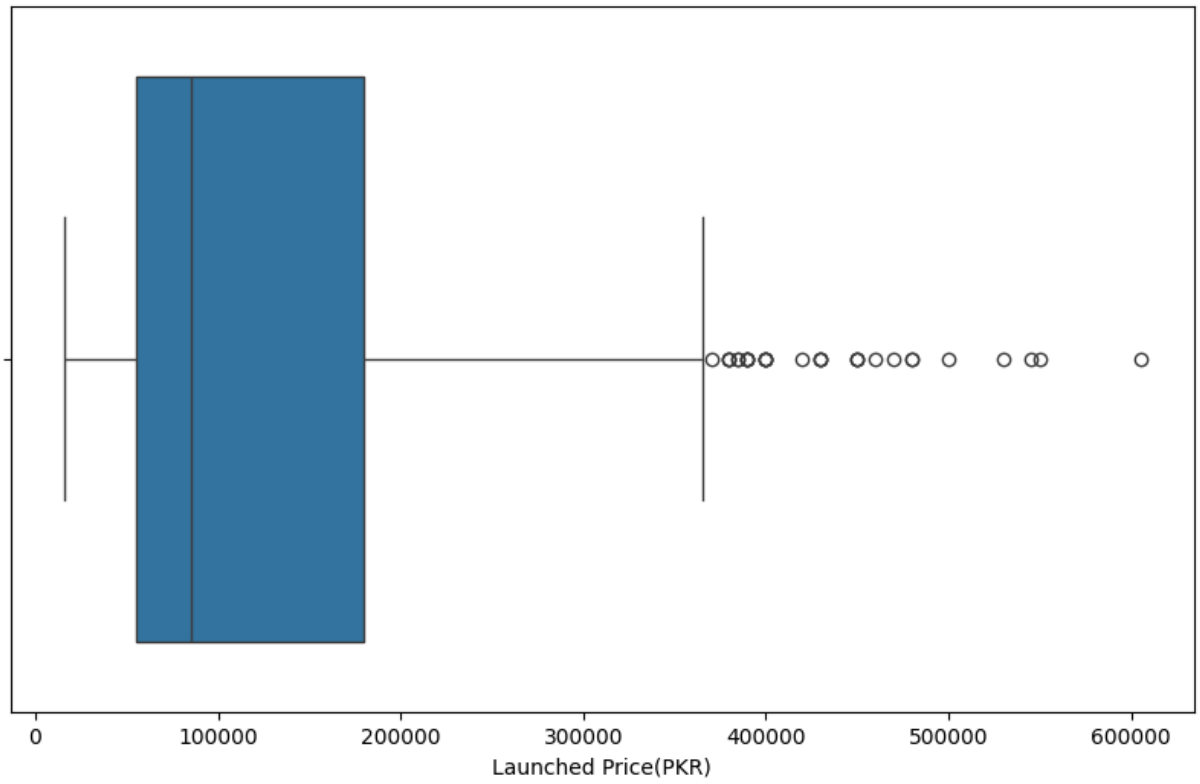
In [438…

```
# Check for the Screen Size(inches)
plt.figure(figsize=(10,6))
sns.boxplot(x='Screen Size(inches)',data=df)
plt.xlabel('Screen Size(inches)')
plt.show()
```



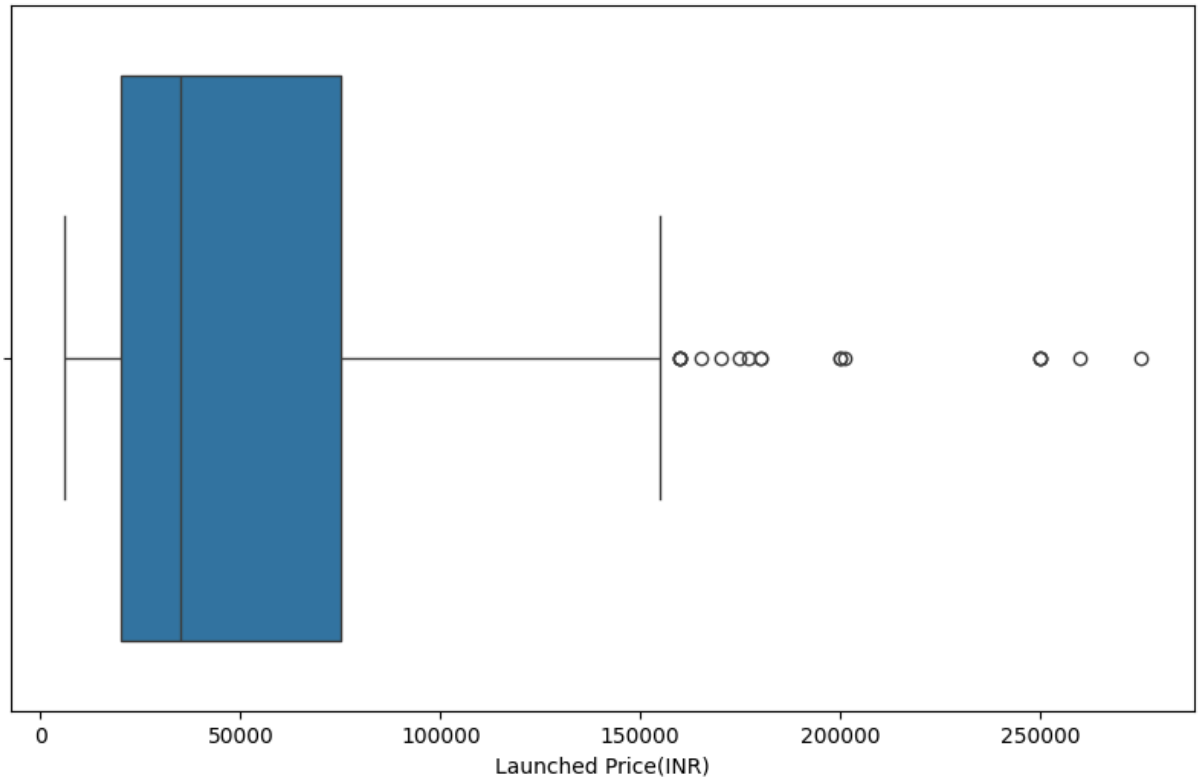
In [439…

```
# Check for the Launched Price(PKR)
plt.figure(figsize=(10,6))
sns.boxplot(x='Launched Price(PKR)',data=df)
plt.xlabel('Launched Price(PKR)')
plt.show()
```



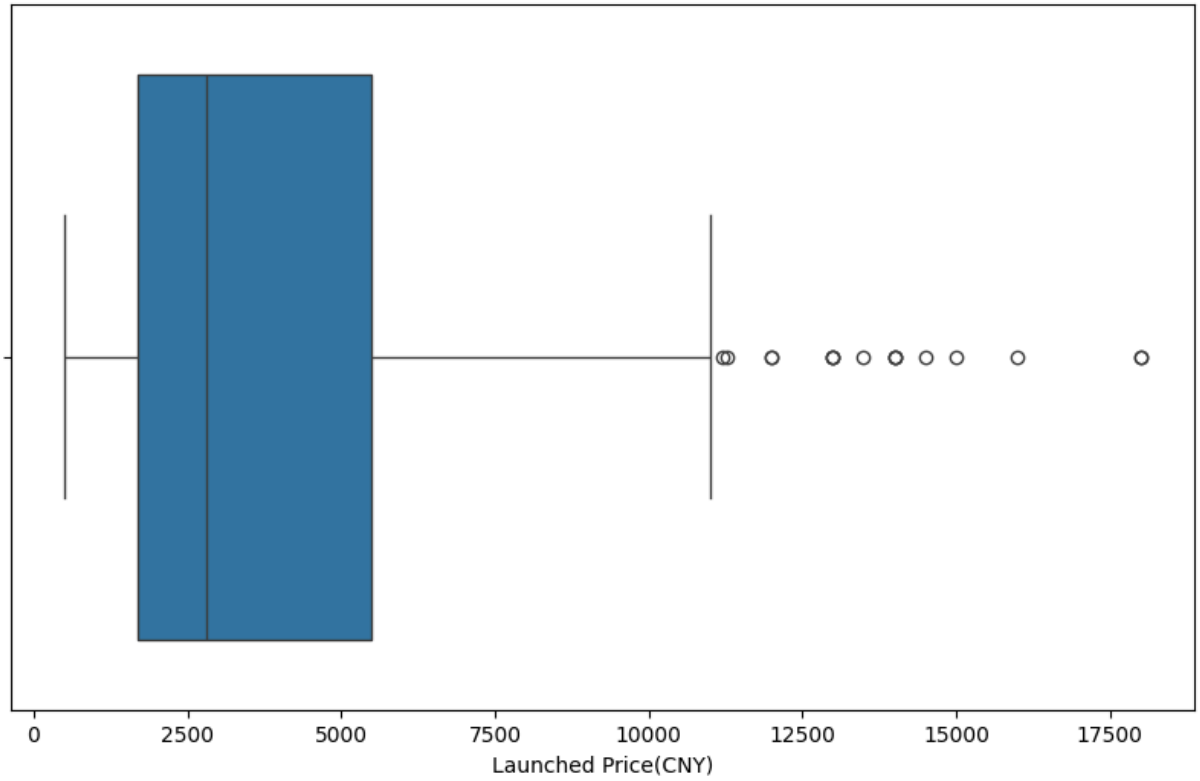
In [440…

```
# Check for the Launched Price(INR)
plt.figure(figsize=(10,6))
sns.boxplot(x='Launched Price(INR)',data=df)
plt.xlabel('Launched Price(INR)')
plt.show()
```



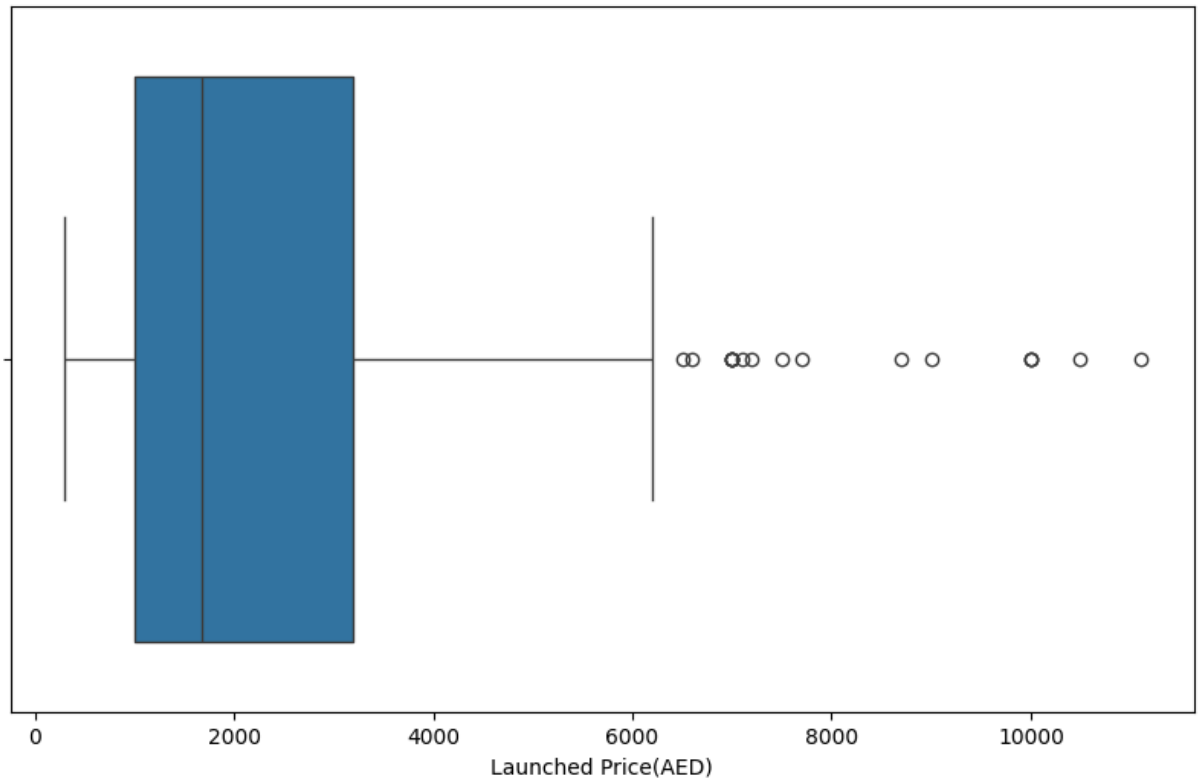
In [441…

```
# Check for the Launched Price(CNY)
plt.figure(figsize=(10,6))
sns.boxplot(x='Launched Price(CNY)',data=df)
plt.xlabel('Launched Price(CNY)')
plt.show()
```



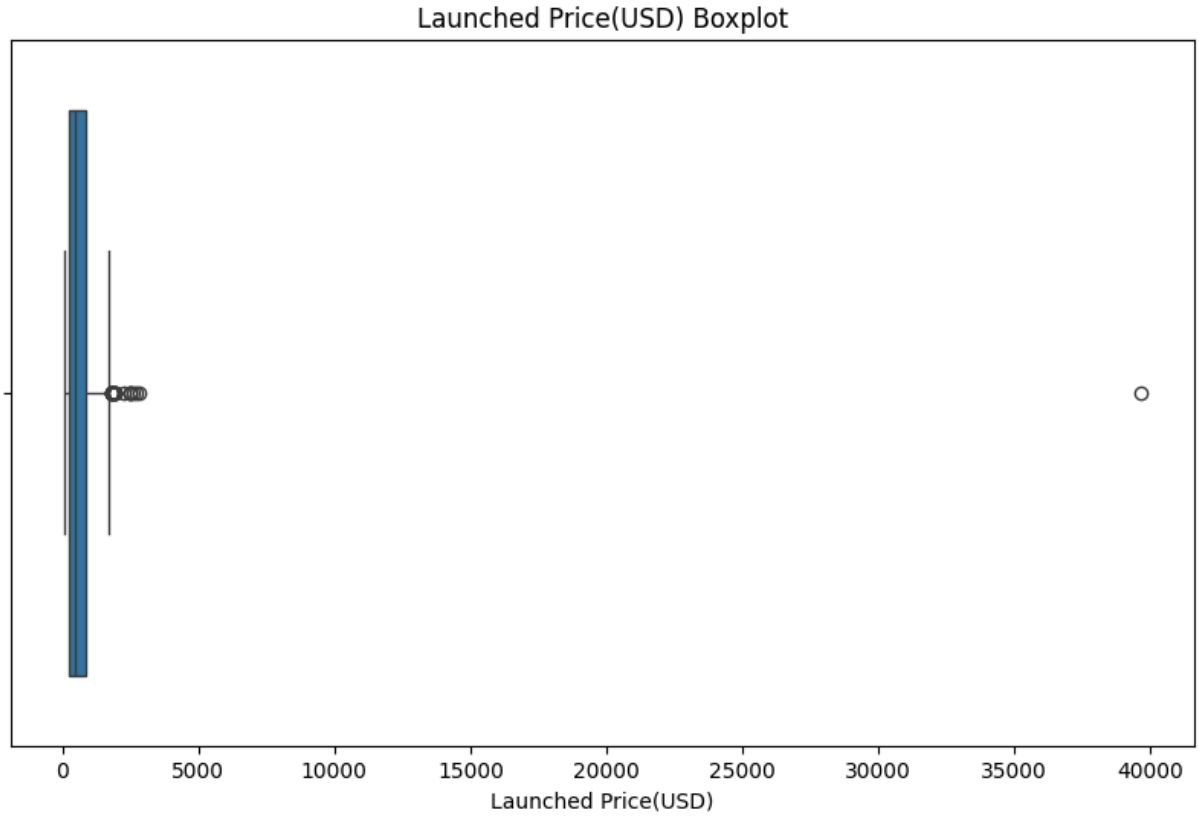
In [442…

```
# Check for the Launched Price(AED)
plt.figure(figsize=(10,6))
sns.boxplot(x='Launched Price(AED)',data=df)
plt.xlabel('Launched Price(AED)')
plt.show()
```



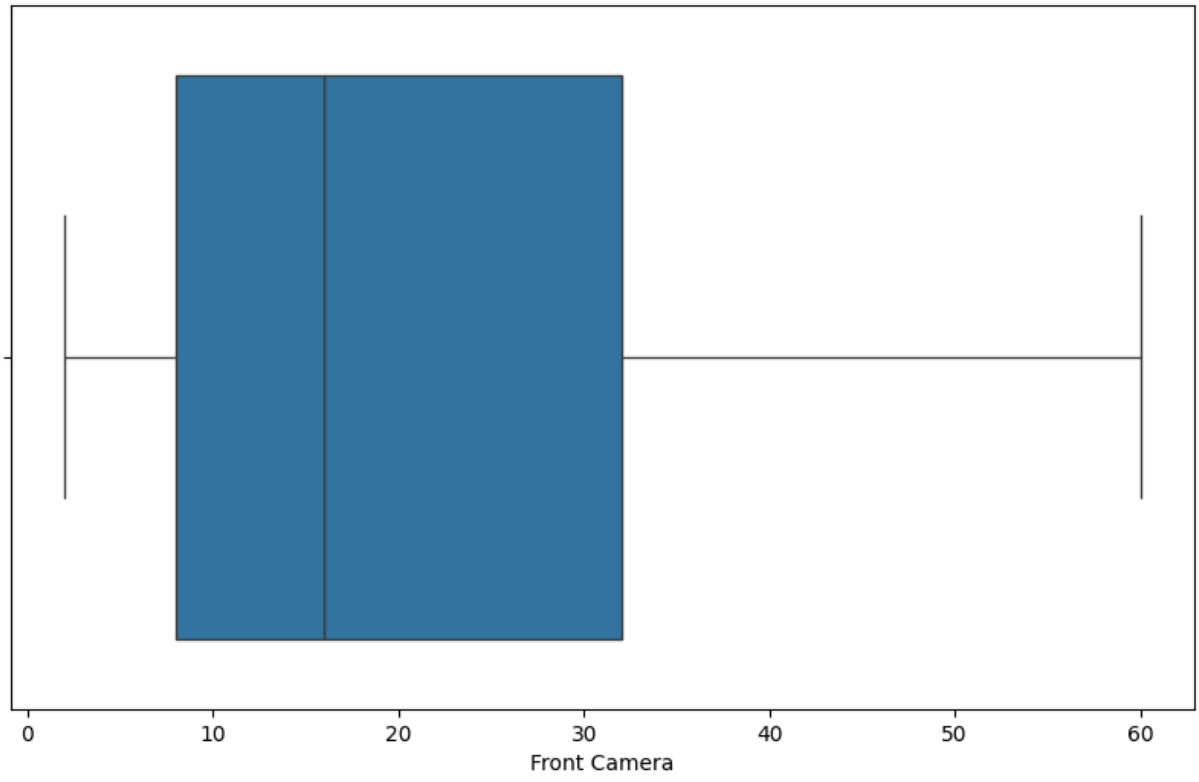
In [443…

```
# Check for the Launched Price(USD)
plt.figure(figsize=(10,6))
sns.boxplot(x='Launched Price(USD)',data=df)
plt.xlabel('Launched Price(USD)')
plt.title('Launched Price(USD) Boxplot')
plt.show()
```



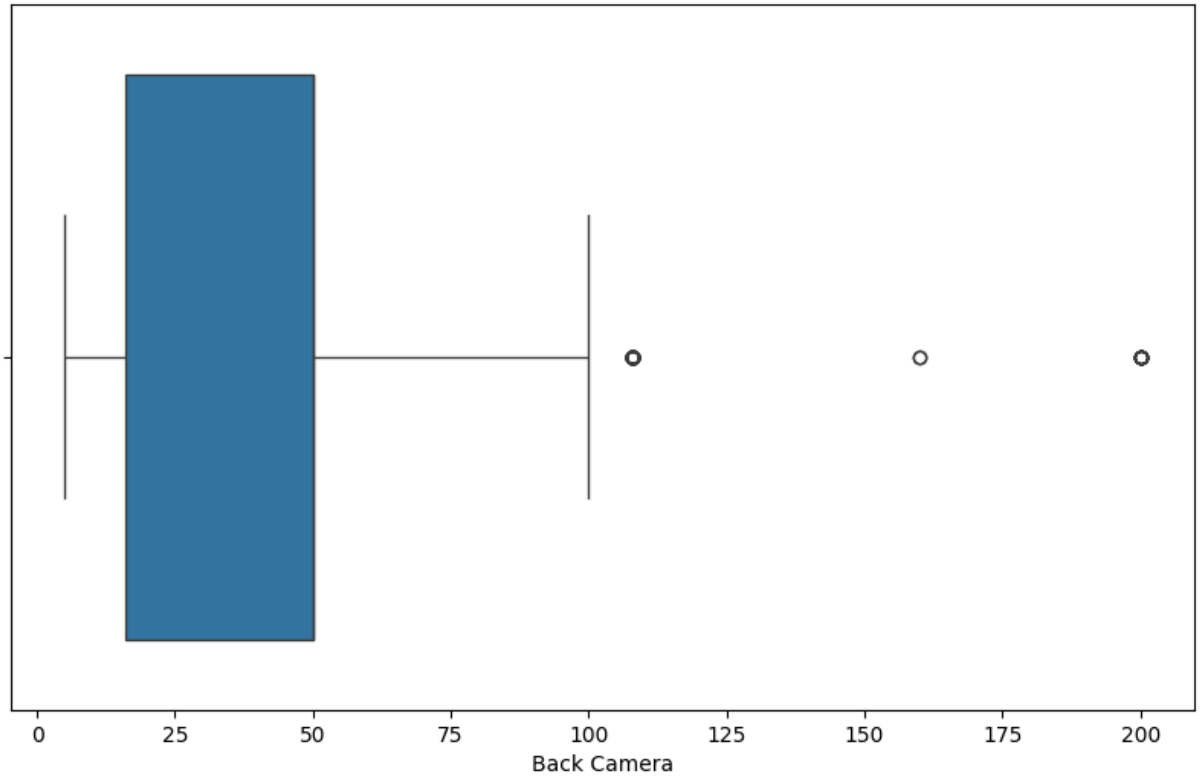
In [444…

```
# Check for the Front Camera
plt.figure(figsize=(10,6))
sns.boxplot(x='Front Camera',data=df)
plt.xlabel('Front Camera')
plt.show()
```



In [445…

```
# Check for the Back Camera
plt.figure(figsize=(10,6))
sns.boxplot(x='Back Camera',data=df)
plt.xlabel('Back Camera')
plt.show()
```



Why don't we remove the outliers in the numeric colomns in the df dataset?

The outliers in the numeric columns are not removed in the df dataset because the outliers are not necessarily incorrect data points.They are valid data points that are just extreme.Removing them could lead to biased result and loss of information.

---

13. Answers the Question Related to the df dataset.

1. What is the most common processor in the dataset?

The most common Processor is **Snapdragon 8 Gen 2**.

2. How many processors have a count of 30?

The number of processors with a count of 30 is **1**.

3. Which processor has the second highest count?

The Processor that has a second highest count is **MediaTek Dimensity 810**.

4. How many processors have a count greater than 20?

The Processor that have a count of more then twenty are **3** in number.

5. What is the count of the A15 Bionic processor?

The count of A15 Bionic Processor is **18**.

6. How many processors have a count of exactly 10?

Processor that have a count of exectly ten is **5** in number.

7. Which processors have a count of 9?

Thr Processor that have a count of nine are:

1. Unisoc T606
2. Exynos 850
3. MediaTek Dimensity 1200
4. Exynos 1380
5. Snapdragon 865

8. What is the total number of unique processors in the dataset?

The total number of unique processors in the dataset is **20**.

9. How many processors have a count less than 5?

Processor that have a count less than fifteen are **158** in number.

10. What is the count of the Snapdragon 8 Gen 1 processor?

The count of Snapdragon 8 Gen 1 processor is **16**.

11. Which processor has the lowest count?

MediaTek Helio P22 has the lowest count.

12. How many processors have a count of exactly 2?

Total no. of count of processor that have a count of exectly 2 is **64** in number.

13. What is the count of the MediaTek Dimensity 810 processor?

The count is **22**.

14. How many processors have a count between 5 and 10?

Number of processor that have a count between five and ten is **37**.

15. What is the count of the Exynos 850 processor?

The count is **9**.

16. How many processors have a count of exactly 1?

The count is **54**.

17. Which processors have a count of 4?

The processor are:

1. Snapdragon 695 5G
2. Snapdragon 7 Gen 1
3. Snapdragon 7 Gen 3
4. Snapdragon 662
5. Qualcomm Snapdragon 778G
6. Snapdragon 888 4G
7. MediaTek Helio G25
8. MediaTek Dimensity 720
9. Qualcomm Snapdragon 768G
10. Snapdragon 6s Gen 3
11. Unisoc T700
12. Exynos 990
13. Exynos 1280
14. Exynos 9825
15. Qualcomm Snapdragon 712
16. Snapdragon 710
17. MediaTek Helio G96
18. Qualcomm Snapdragon 675
19. Dimensity 1200
20. Qualcomm Snapdragon 8 Gen 1
21. Google Tensor G4
22. Qualcomm Snapdragon 8+ Gen 1

18. What is the count of the Snapdragon 888 processor?

The count is **8**.

19. How many processors have a count of exactly 3?

18 processors have a count of exactly 3.

20. What is the count of the MediaTek Helio G85 processor?

19 processors have a count of MediaTek Helio G85.

21. How many processors have a count of exactly 6?

8 processors have a count of exactly 6.

22. Which processors have a count of 7?

Thr processor that have a count of exectly seven is:

1. Snapdragon 855
2. MediaTek Helio G37
3. MediaTek Dimensity 6020
4. Snapdragon 6s 4G Gen 1

23. What is the count of the Snapdragon 765G processor?

11 processors have a count of Snapdragon 765G.

24. How many processors have a count of exactly 8?

8 processors have a count of exactly 8.

25. What is the count of the MediaTek Dimensity 9000 processor?

11 processors have a count of MediaTek Dimensity 9000.

26. How many processors have a count of exactly 11?

8 processors have a count of exactly 11.

27. Which processors have a count of 12?

Snapdragon 8+ Gen 1 have a count of 12.

28. What is the count of the Snapdragon 870 processor?

10 processors have a count of Snapdragon 870.

29. How many processors have a count of exactly 14?

3 processors have a count of exactly 14.

30. What is the count of the MediaTek Dimensity 8200 processor?

14 processors have a count of MediaTek Dimensity 8200.

31. What is the average megapixel count for front cameras across all mobile phones in the dataset?

18.16301075268817 is the average megapixel count for front cameras across all mobile phones.

32. How does the average megapixel count of back cameras compare to that of front cameras?

Average Back Camera MP: 46.85462365591398
Average Front Camera MP: 18.16301075268817
The average megapixel count of back cameras is higher than that of front cameras.

33. What is the correlation between the front camera megapixel count and the launched price in USD?

-0.00392445810643407 is the correlation between the front camera megapixel count and the launched price in USD, and the result show that there is a weak negative correlation between the two variables.